สยามชำนาญกิจ
SIAM CHAMNAN KIT

Mobile Testing with Appium + Playwright TypeScript

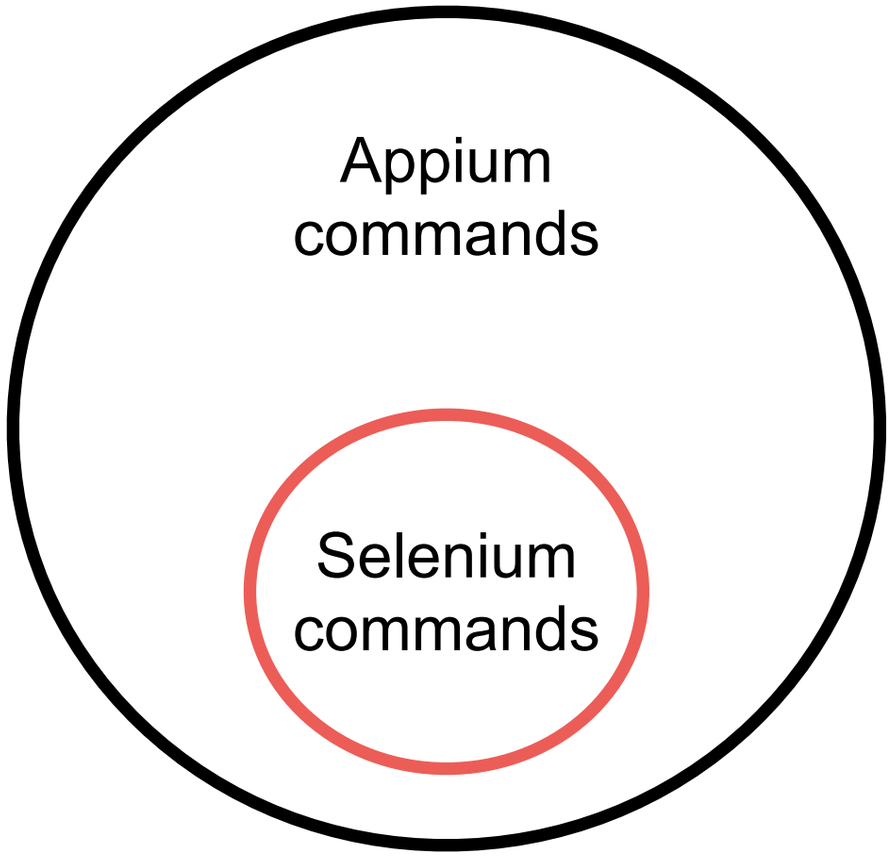
Automated Testing

https://appium.io/

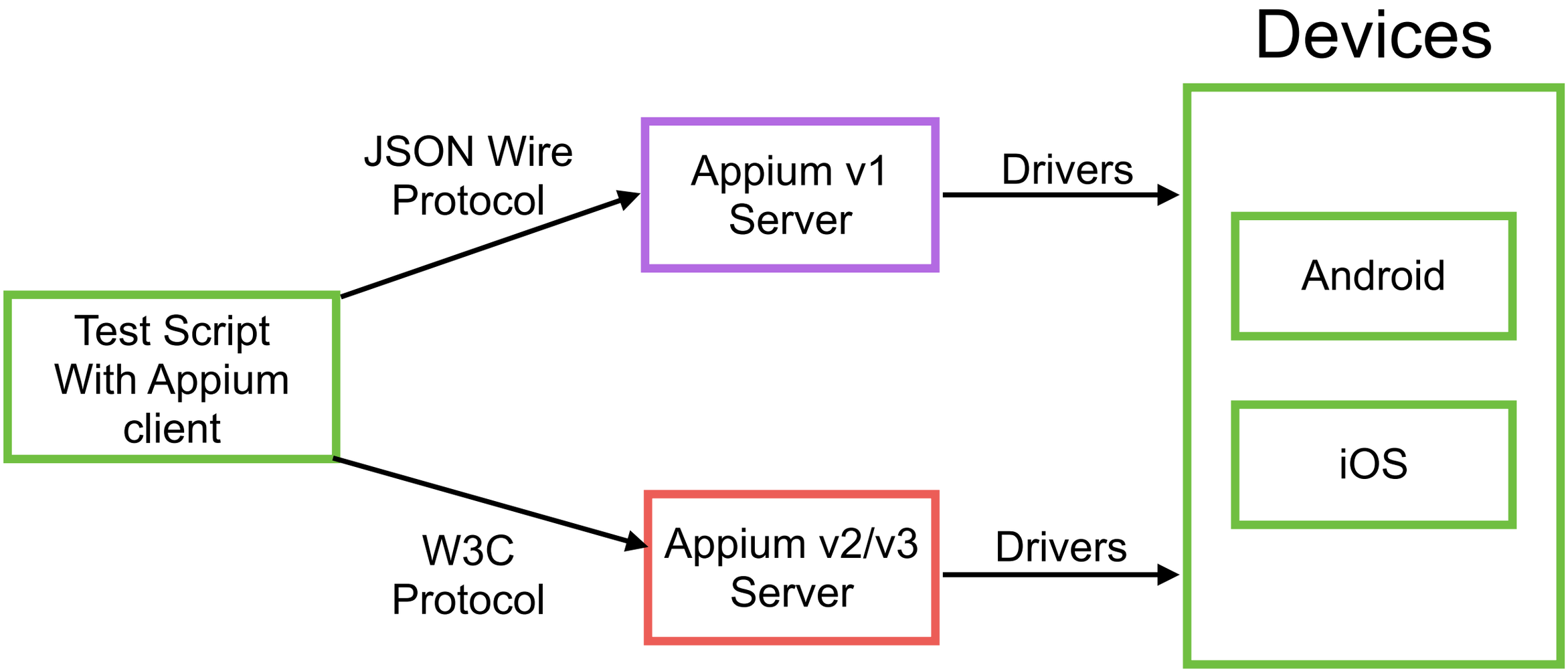
Automated Testing

Appium

Automation tool for mobile app
Selenium for mobile
High compatibility with selenium



Architecture



Appium Tools

Appium Server via npm

Appium Inspector

Android	iOS	Web	Desktop

https://appium.io/docs/en/latest/quickstart/

Appium Drivers

Platform	Driver	Platform Versions	Appium Version
iOS	XCUITest	9.3+	1.6.0+
	UIAutomation	8.0 to 9.3	All
Android	Espresso	?+	1.9.0+
	UiAutomator2	?+	1.6.0+
	UiAutomator	4.3+	All
Mac	Mac	?+	1.6.4+
Windows	Windows	10+	1.6.0+

https://appium.io/docs/en/latest/ecosystem/drivers/

Appium Clients

JavaScript/TypeScript

Python

Java

C#

Robotframework

https://appium.io/docs/en/latest/ecosystem/clients/

Let’s start with Appium

Appium doctor (android, iOS)

appium™

Steps to run

Appium doctor and drivers
Start server
Provide target apps (api, ipa/app)
Appium Inspector

Check environments

Android	iOS
APPIUM_HOME	APPIUM_HOME
ANDROID_HOME	Xcode
JAVA_HOME	Xcode command line tools
adb, emulator, apkanalyzer	ios-deploy, applesimutils
NodeJS	NodeJS

List of android’s devices

$adb devices

List of iOS's devices

$xcrun xctrace list devices

```
== Simulators ==
Apple TV Simulator (15.2) (9666637B-AD71-47CD-932D-DE7BA9096F46)
Apple TV 4K (2nd generation) Simulator (15.2) (2B6A15B3-4A03-41B5-B6C6-D7727DCA94D8)
Apple TV 4K (at 1080p) (2nd generation) Simulator (15.2) (EC5B5310-9DE7-4CB2-8AE2-172FC885DB8C)
iPad (9th generation) Simulator (15.2) (03B84395-1A2B-4759-B01A-171CD4F8F796)
iPad Air (4th generation) Simulator (15.2) (6389A3CA-7805-452D-9110-5B3796A9D9BB)
iPad Pro (11-inch) (3rd generation) Simulator (15.2) (40E522A5-14CE-453C-A54C-29D096F19236)
iPad Pro (12.9-inch) (5th generation) Simulator (15.2) (1E6E63EF-FA56-4E31-8082-13B0C664AD33)
iPad Pro (9.7-inch) Simulator (15.2) (9C7181D9-73D6-48BF-8972-80A73ADB2EA9)
iPad mini (6th generation) Simulator (15.2) (01F8F0AB-1AF5-4FBE-9587-B7713F7BBC6F)
iPhone 11 Simulator (15.2) (1E75E325-57EC-4D6D-85BF-9958B0A9B158)
iPhone 11 Pro Simulator (15.2) (7D50F81C-3521-447F-A79C-8613C6C58FEF)
iPhone 11 Pro Max Simulator (15.2) (FB336CEE-A241-43D1-8704-B0632EA9FD28)
iPhone 12 Simulator (15.2) (5325CF3D-9576-462A-A110-49A5D76665FE)
```

List of Appium driver

$appium driver list

```
✔ Listing available drivers (rerun with --verbose for more info)
- uiautomator2@7.0.0 [installed (npm)]
- xcuitest [not installed]
- espresso [not installed]
- mac2 [not installed]
- windows [not installed]
- safari [not installed]
- gecko [not installed]
- chromium [not installed]
```

https://appium.io/docs/en/latest/ecosystem/drivers/

Appium Server

Appium Server

Install via npm (required Node.js)

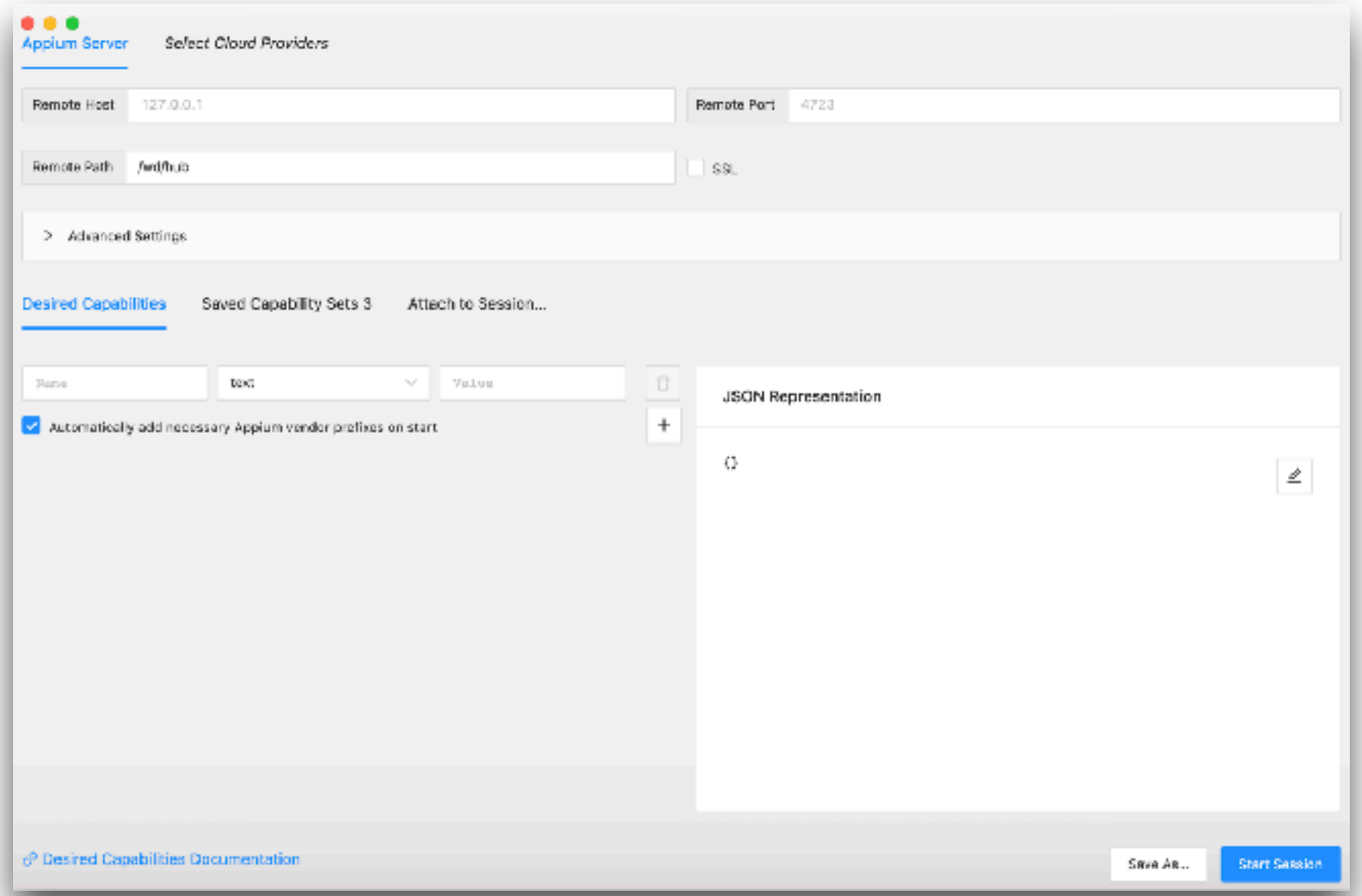
```
$npm install -g appium
$appium
```

https://github.com/appium/appium

Appium Inspector

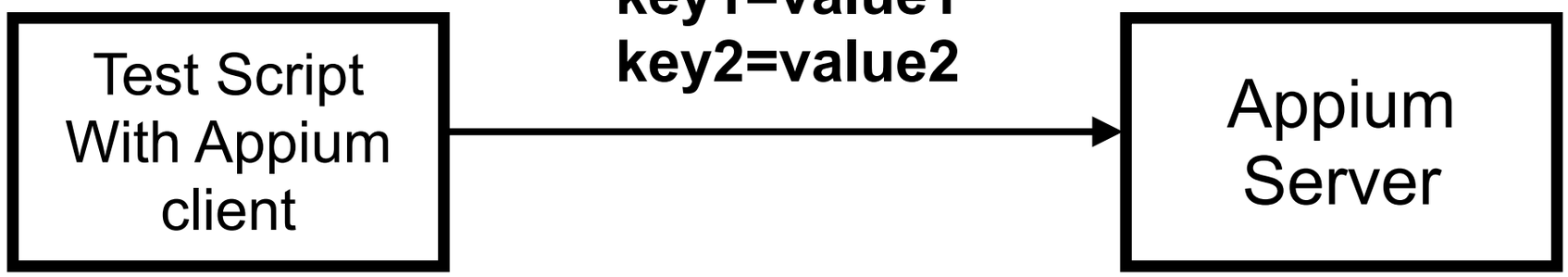
Appium Inspector

GUI inspector for mobile apps



https://github.com/appium/appium-inspector

Create a session with capabilities



http://appium.io/docs/en/writing-running-appium/caps/index.html

Required capabilities

Name	Description
platformName	Platform to automate (iOS, Android)
platformVersion	Version of platform
deviceName	Type of device to automate
app	Path to your app

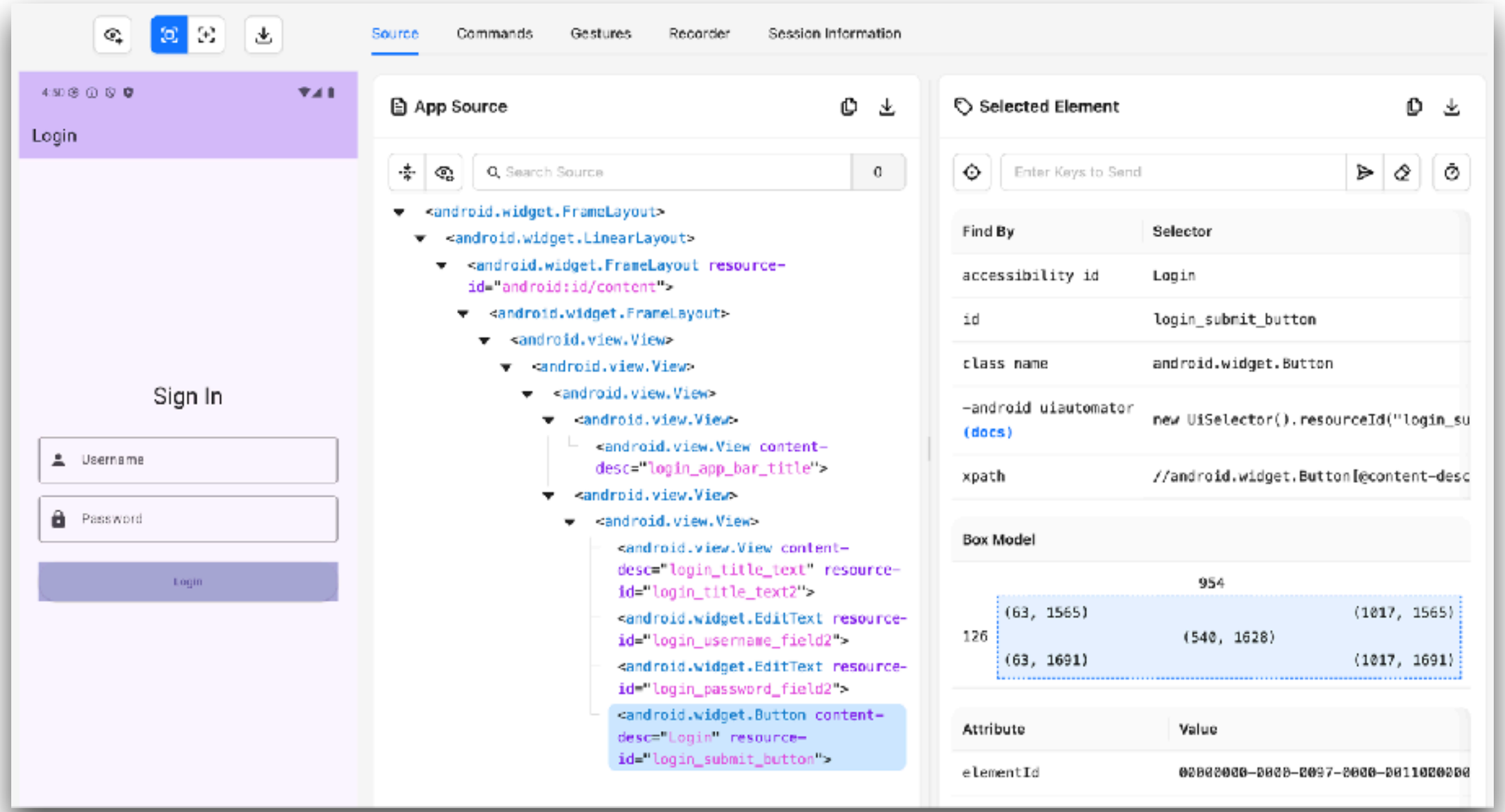
https://appium.io/docs/en/3.2/guides/caps/

Session capabilities for android

```
{
  "platformName": "Android",
  "appium:automationName": "UiAutomator2",
  "appium:appPackage": "com.demo.registerapp",
  "appium:deviceName": "R58M36QKSJK"
}
```

https://github.com/appium/appium-uiautomator2-driver?tab=readme-ov-file#capabilities

Appium Inspector (Android)

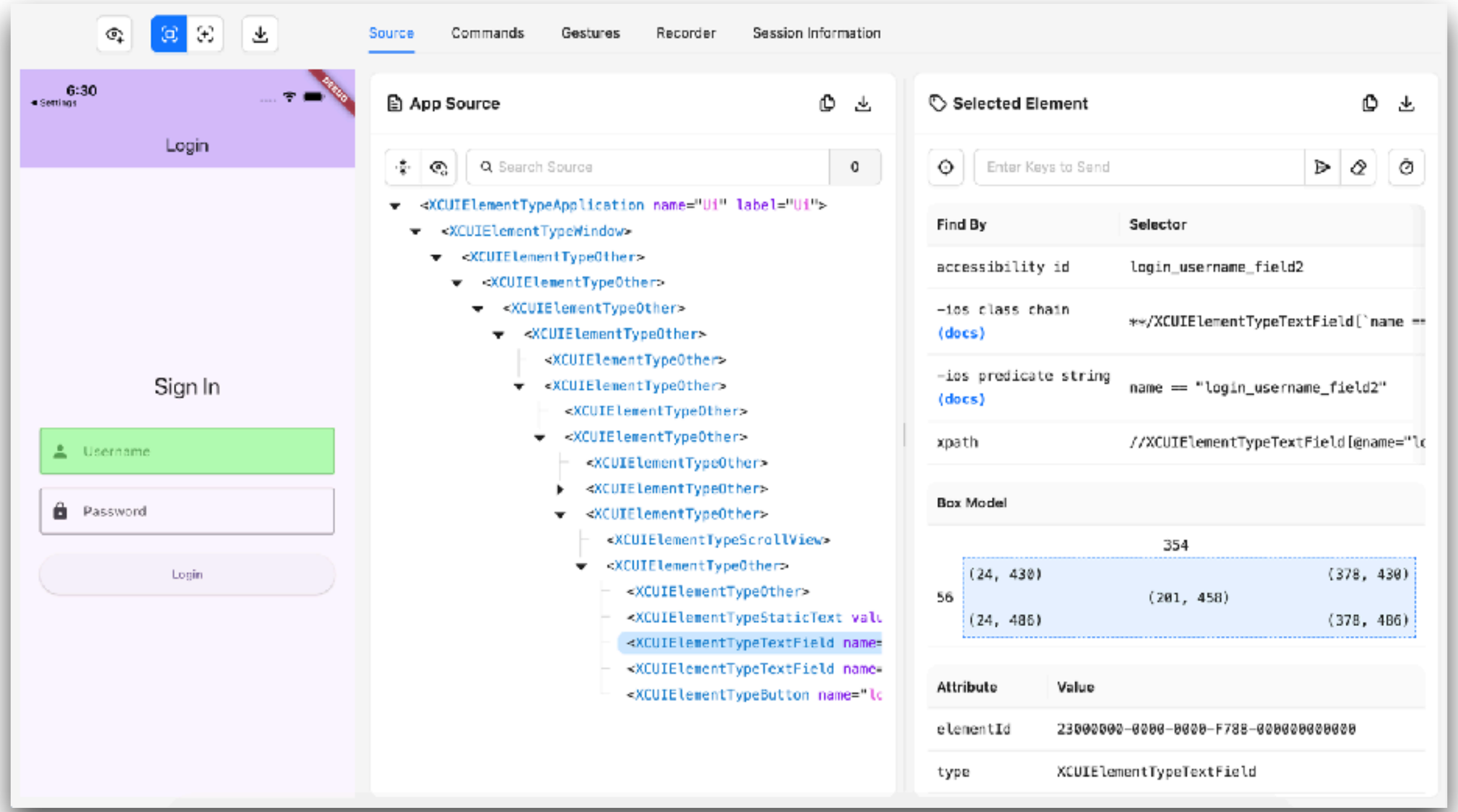


Session capabilities for iOS

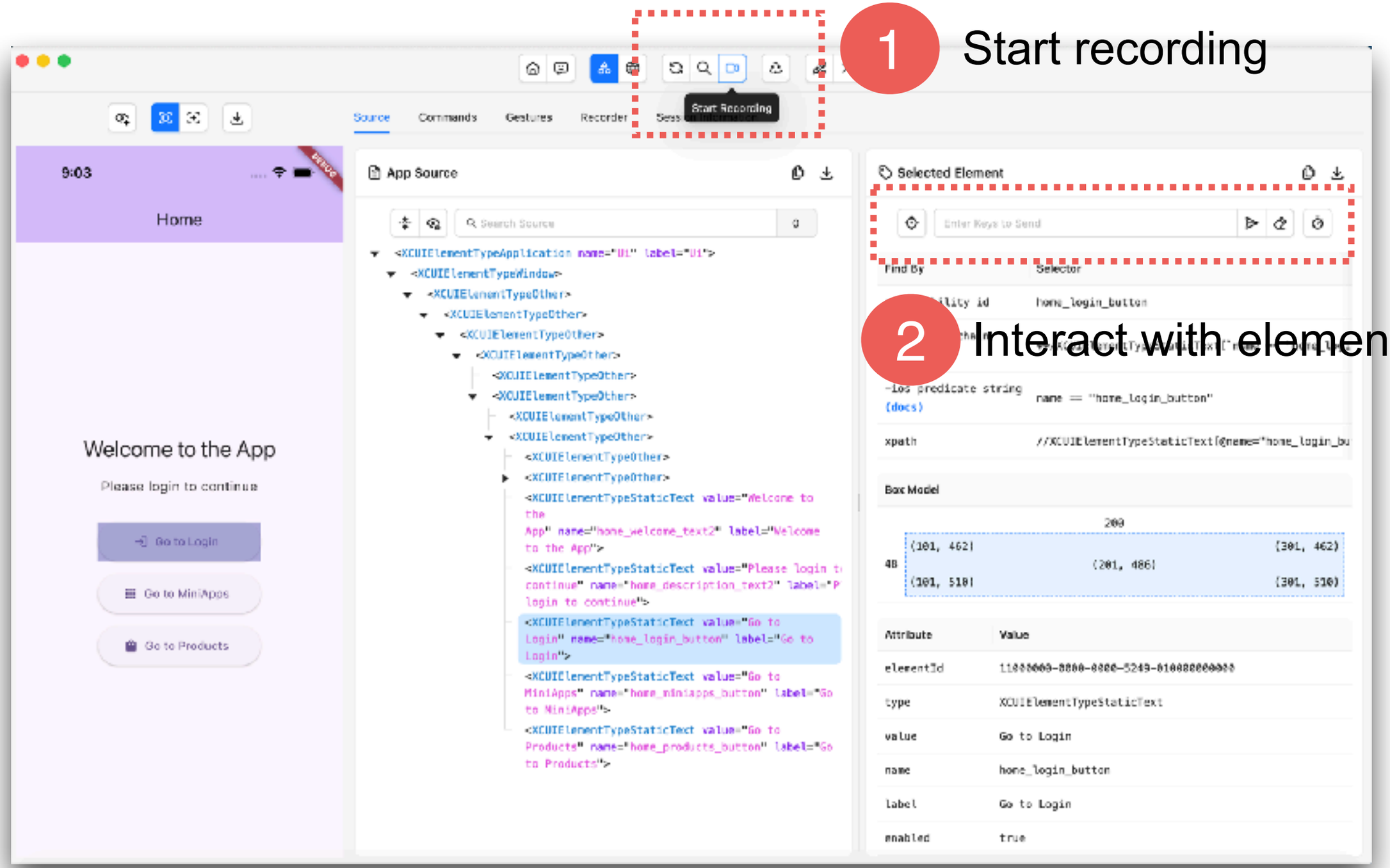
```
{
  "platformName": "iOS",
  "appium:deviceName": "iPhone 17",
  "appium:platformVersion": "26.2",
  "appium:automationName": "XCUITest",
  "appium:bundleId": "com.example.ui",
  "appium:connectionRetryTimeout": 60000,
  "appium:noReset": true
}
```

https://appium.github.io/appium-xcuitest-driver/latest/reference/capabilities/

Appium Inspector (iOS)

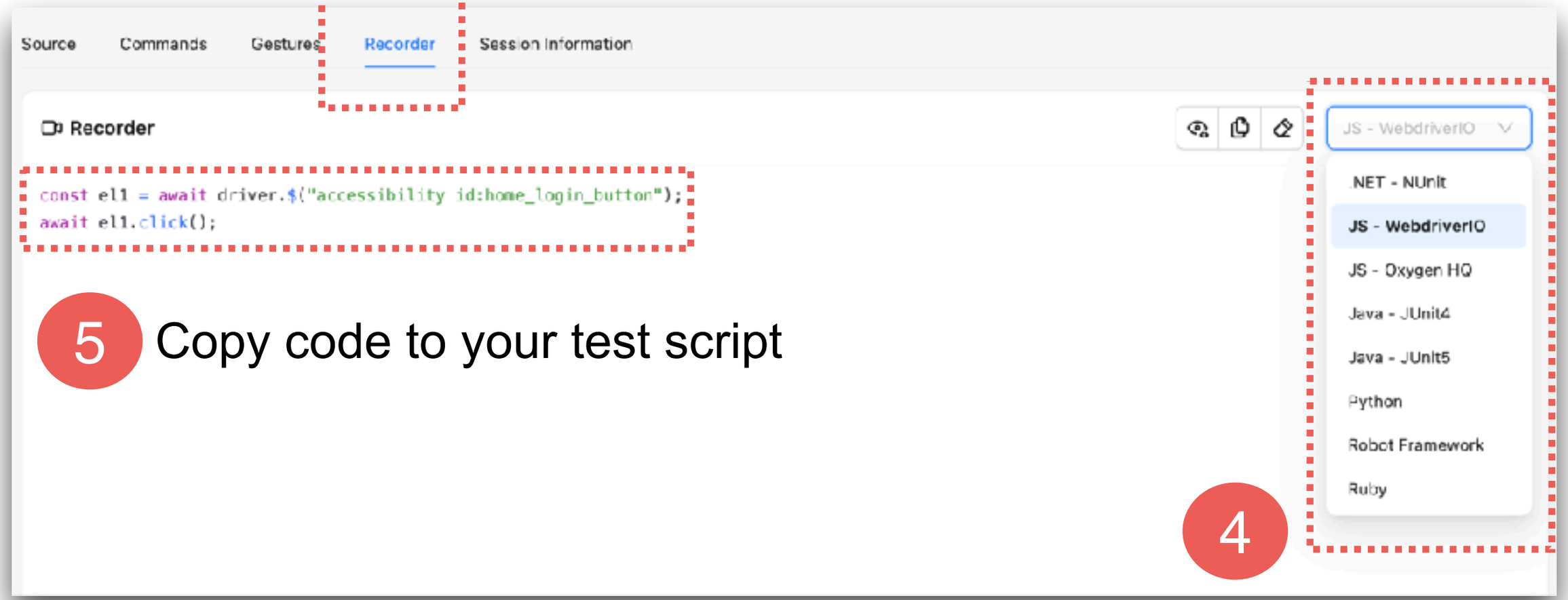


Record from Appium Inspector



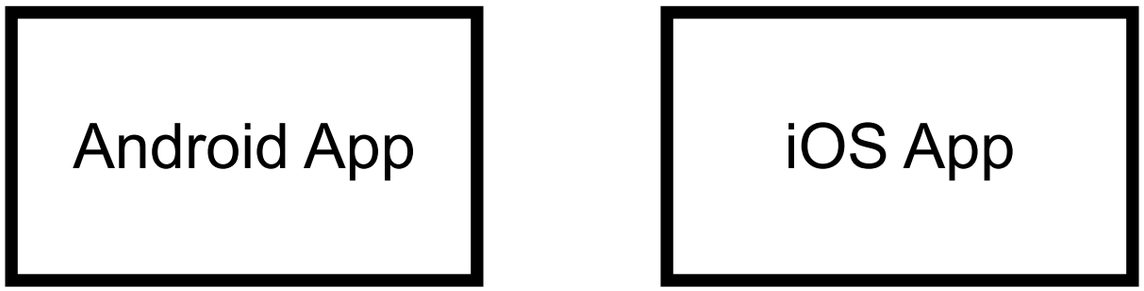
Record from Appium Inspector

3 Go to tab Recorder



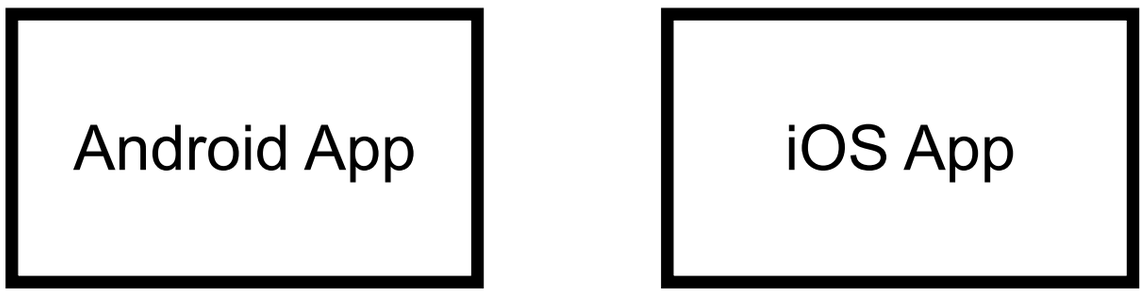
Choose JS-WebdriverIO

Appium with Flutter App



Finding and Using Elements

Isolated from UI !!

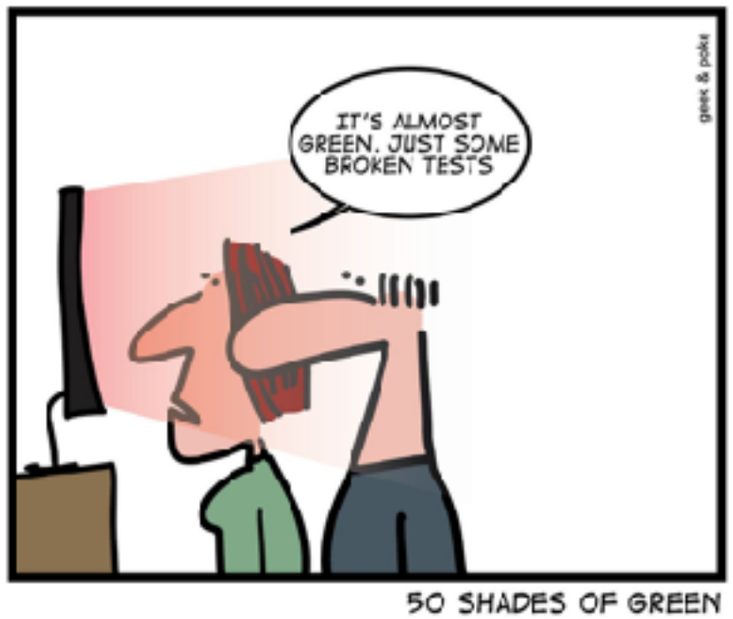


Locator Strategies

Strategy	Description
Accessibility ID	iOS = accessibility-id Android = content-desc
ID	iOS = name Android = resource-id
Name	Name of element
XPath	Pattern of element in XML (not recommended) Absolute path vs Relative path ?
Android UIAutomator	Android only
iOS Predicate String	iOS only

Good Locators

Unique
Descriptive
Resilient
Shorter in length (maintainability)



Flutter App Locator

Key (default)

Text

Semantic identifier

Tooltip

Type

Working with Semantics widget

Use identifier in Flutter 3.19+

Android App

iOS App

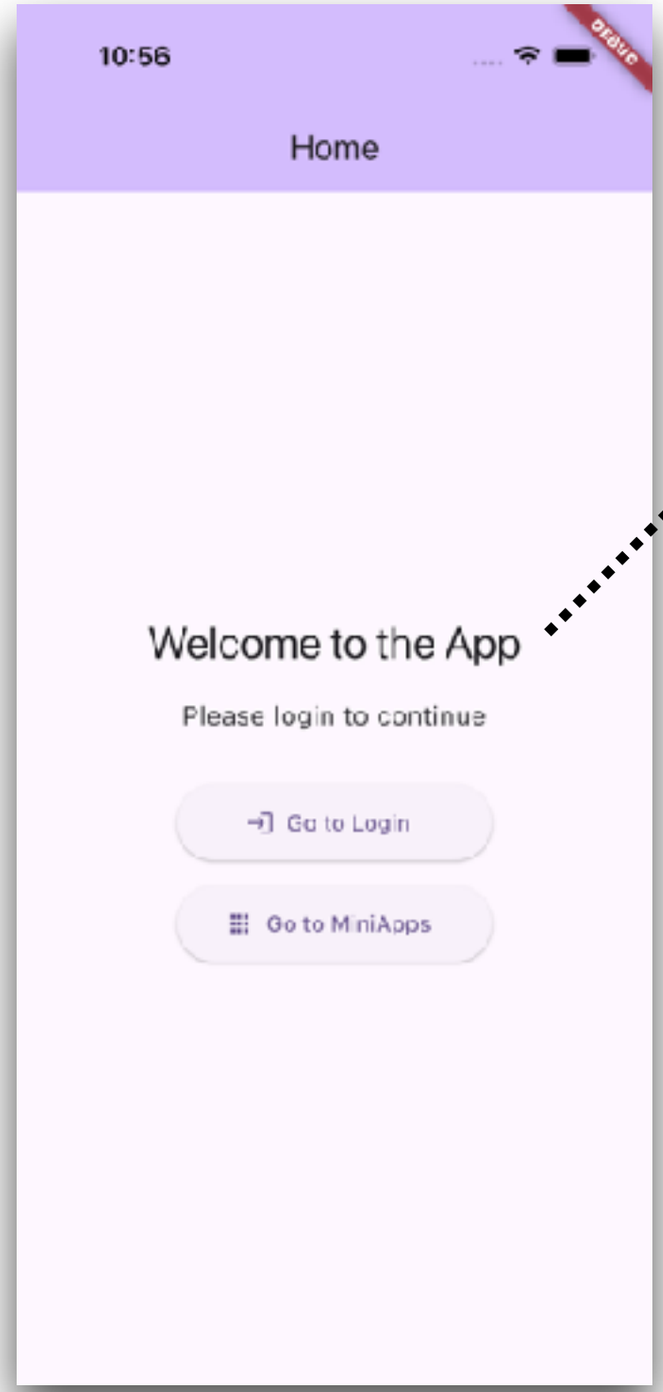
https://blog.flutter.dev/whats-new-in-flutter-3-19-58b1aae242d2

SemanticsProperties accessibility identifier

A new accessibility identifier in `SemanticsProperties` provides an identifier for the semantic node in the native accessibility hierarchy. On Android, it appears in the accessibility hierarchy as `resource-id`. On iOS, this sets `UIAccessibilityElement.accessibilityIdentifier`. We want to thank community member @bartekpacia for this change, which spanned the engine and framework.

https://blog.flutter.dev/whats-new-in-flutter-3-19-58b1aae242d2

Text (iOS app)



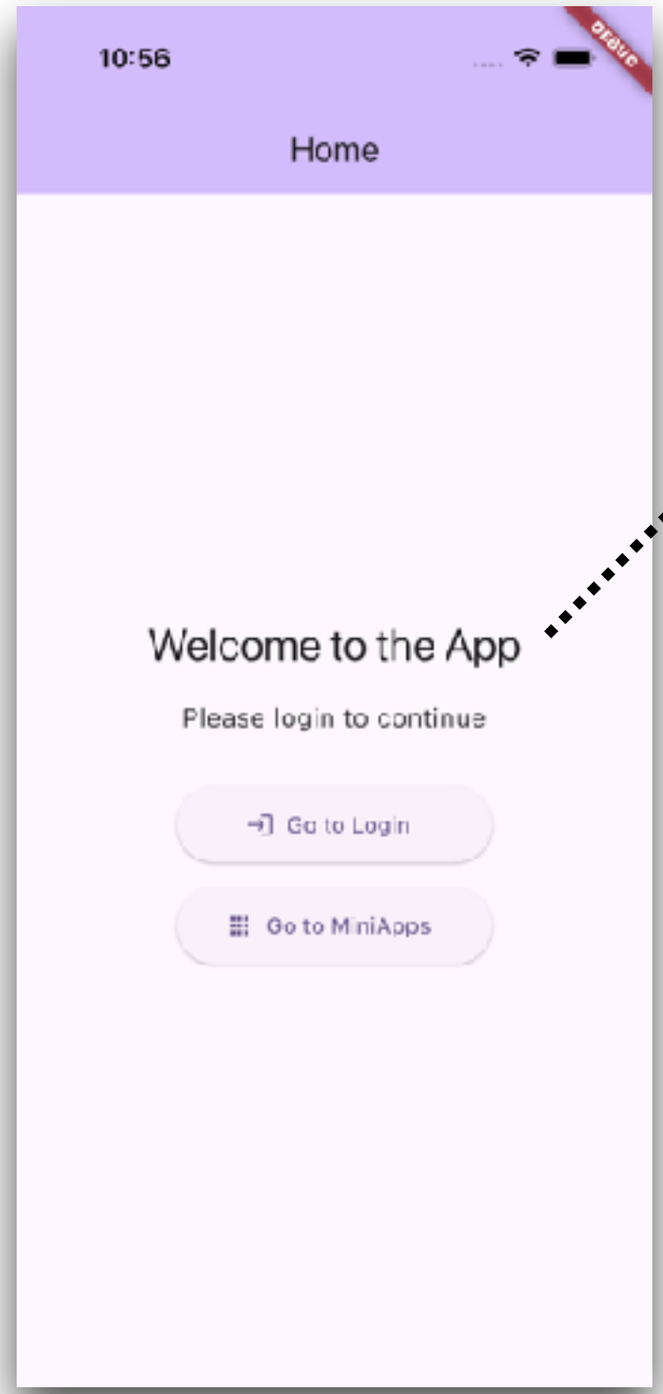
Flutter code

```dart
Text(
    'Welcome to the App',
    key: const Key('home_welcome_text'),
    semanticsIdentifier: 'home_welcome_text2',
)
```

Appium inspector (accessibility, text)

Find By	Selector
accessibility id	home_welcome_text2
-ios class chain (docs)	**/XCUIElementTypeStaticText[`name =
-ios predicate string (docs)	name == "home_welcome_text2"
xpath	//XCUIElementTypeStaticText[@name="h

Text (Android app)



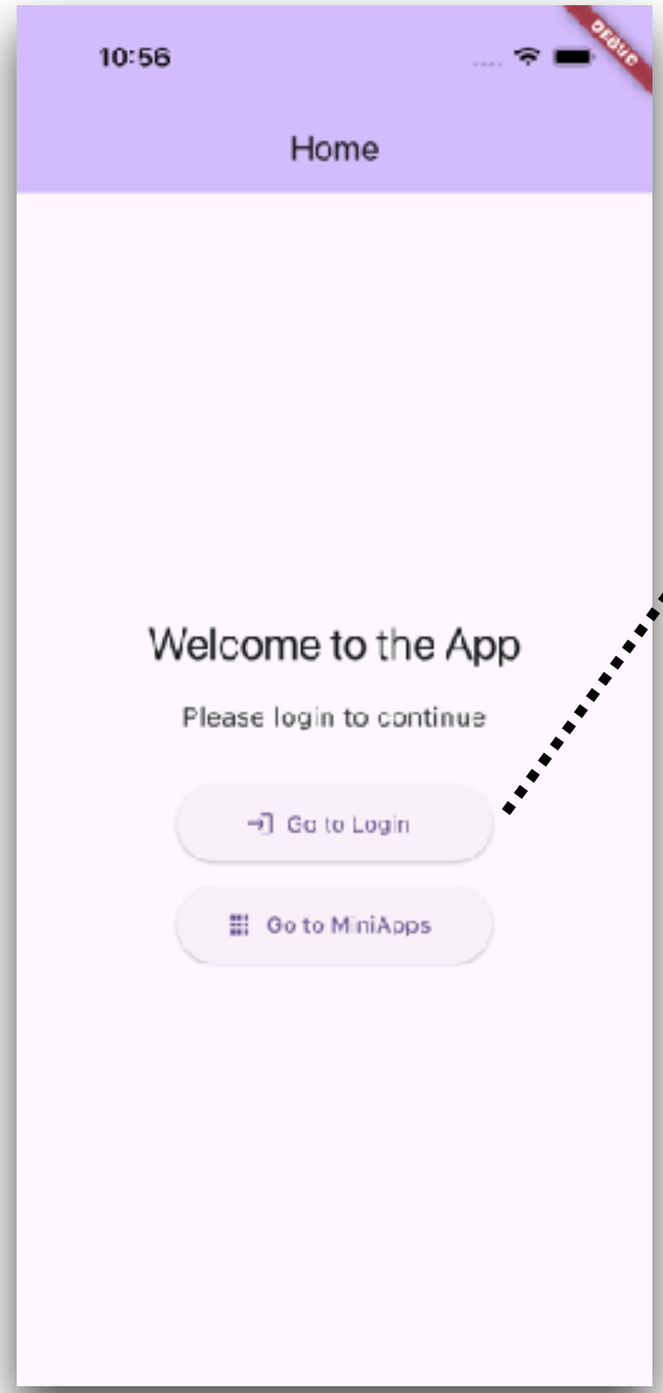
Flutter code

```
Text(
    'Welcome to the App',
    key: const Key('home_welcome_text'),
    semanticsIdentifier: 'home_welcome_text2',
)
```

Appium inspector (resource-id, content-desc)

Find By	Selector
accessibility id	Welcome to the App
id	home_welcome_text2
-android uiautomator (docs)	new UiSelector().resourceId("home_wel
xpath	//android.view.View[@content-desc="We

Button (iOS app)



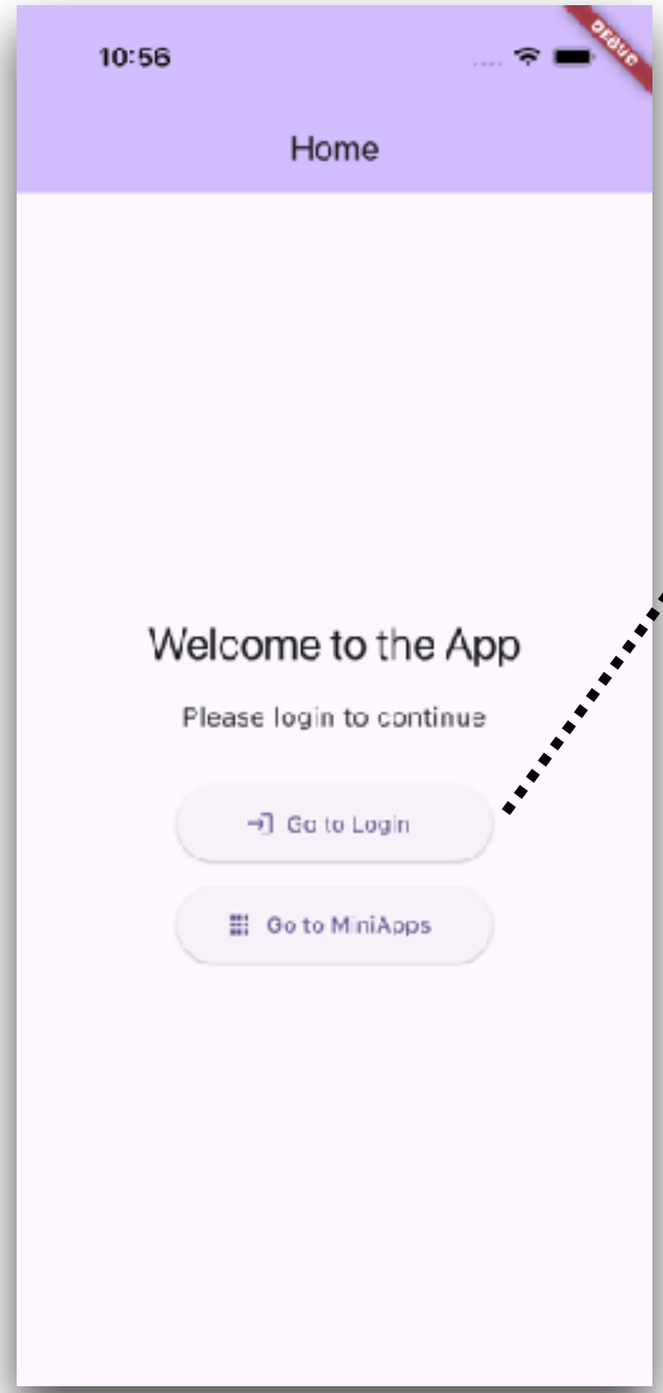
Flutter code

```dart
Semantics(
  identifier: 'home_login_button',
  label: 'Go to Login',
  excludeSemantics: true,
  child: ElevatedButton.icon(
    key: const Key('home_login_button'),
    label: const Text('Go to Login'),
  ),
)
```

Appium inspector (accessibility, value)

Find By	Selector
accessibility id	home_login_button
-ios class chain (docs)	**/XCUIElementTypeStaticText[`name =
-ios predicate string (docs)	name == "home_login_button"
xpath	//XCUIElementTypeStaticText[@name="H

Button (Android app)



Flutter code

```dart
Semantics(
  identifier: 'home_login_button',
  label: 'Go to Login',
  excludeSemantics: true,
  child: ElevatedButton.icon(
    key: const Key('home_login_button'),
    label: const Text('Go to Login'),
  ),
)
```

Appium inspector (resource-id, content-desc)

Find By	Selector
accessibility id	Go to Login
id	home_login_button
-android uiautomator (docs)	new UiSelector().resourceId("home_log
xpath	//android.view.View[@content-desc="Go

ListView (iOS app)

Flutter code

```
return ListView.builder(
  key: const Key('miniapps_list'),
  itemCount: _miniApps!.length,
  itemBuilder: (context, index) {
    final miniApp = _miniApps![index];
    return Semantics(
      identifier: 'miniapp_list',
      container: true,
      child: Card(
        key: Key('miniapp_card_${miniApp.id}'),
```

Appium inspector (accessibility)

ListView (Android app)

Flutter code

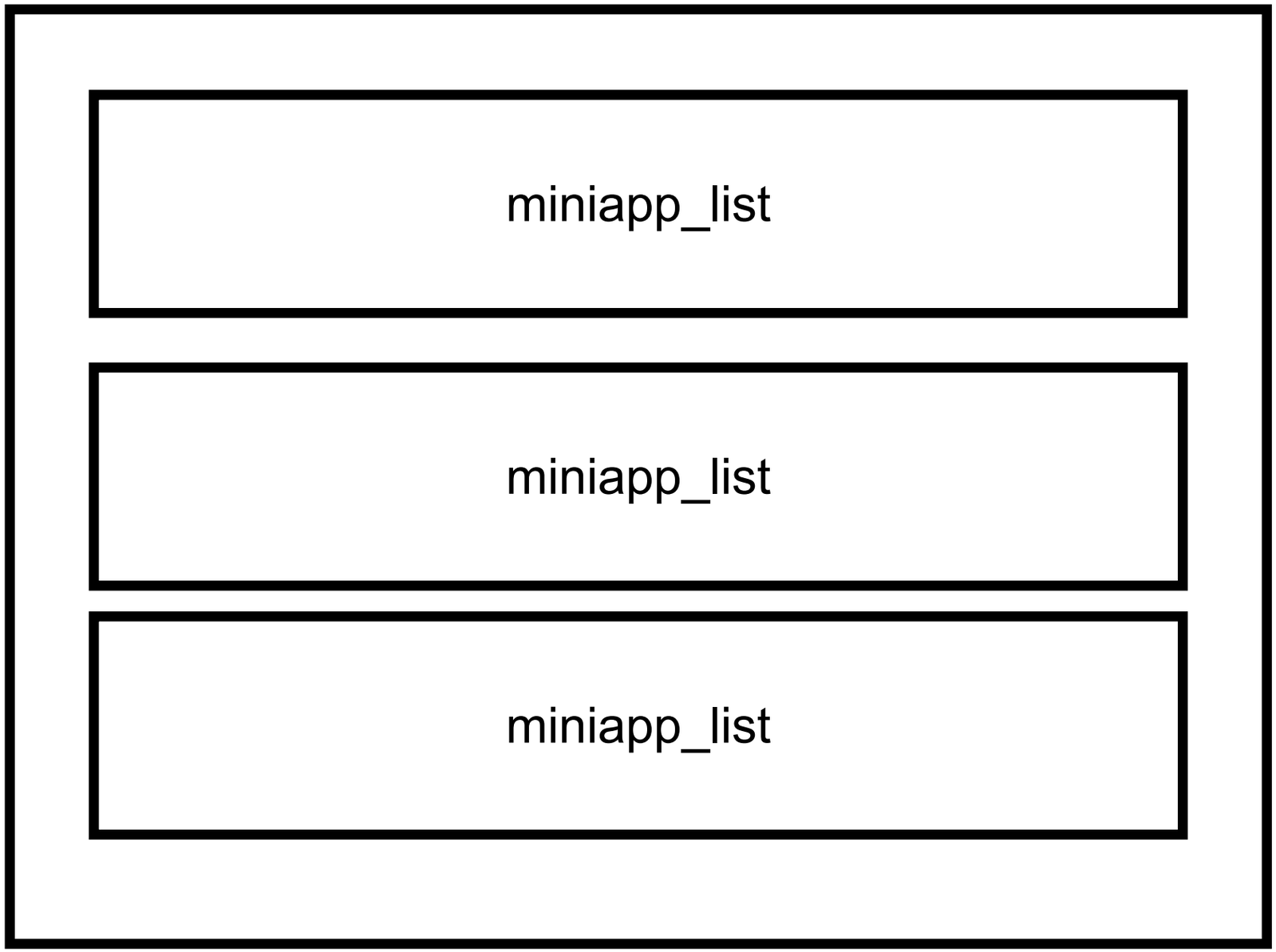
```dart
return ListView.builder(
  key: const Key('miniapps_list'),
  itemCount: _miniApps!.length,
  itemBuilder: (context, index) {
    final miniApp = _miniApps![index];
    return Semantics(
      identifier: 'miniapp_list',
      container: true,
      child: Card(
        key: Key('miniapp_card_${miniApp.id}'),
```

Appium inspector (resource-id)

```
<android.view.View>
  <android.view.View resource-id="miniapp_list">
    <android.widget.Button content-desc="1\nMiniApp
    1\nhttps://miniapp1.example.com" resource-
    id="miniapp_name_1">
  <android.view.View>
    <android.view.View resource-id="miniapp_list">
```

ListView

Size = 3



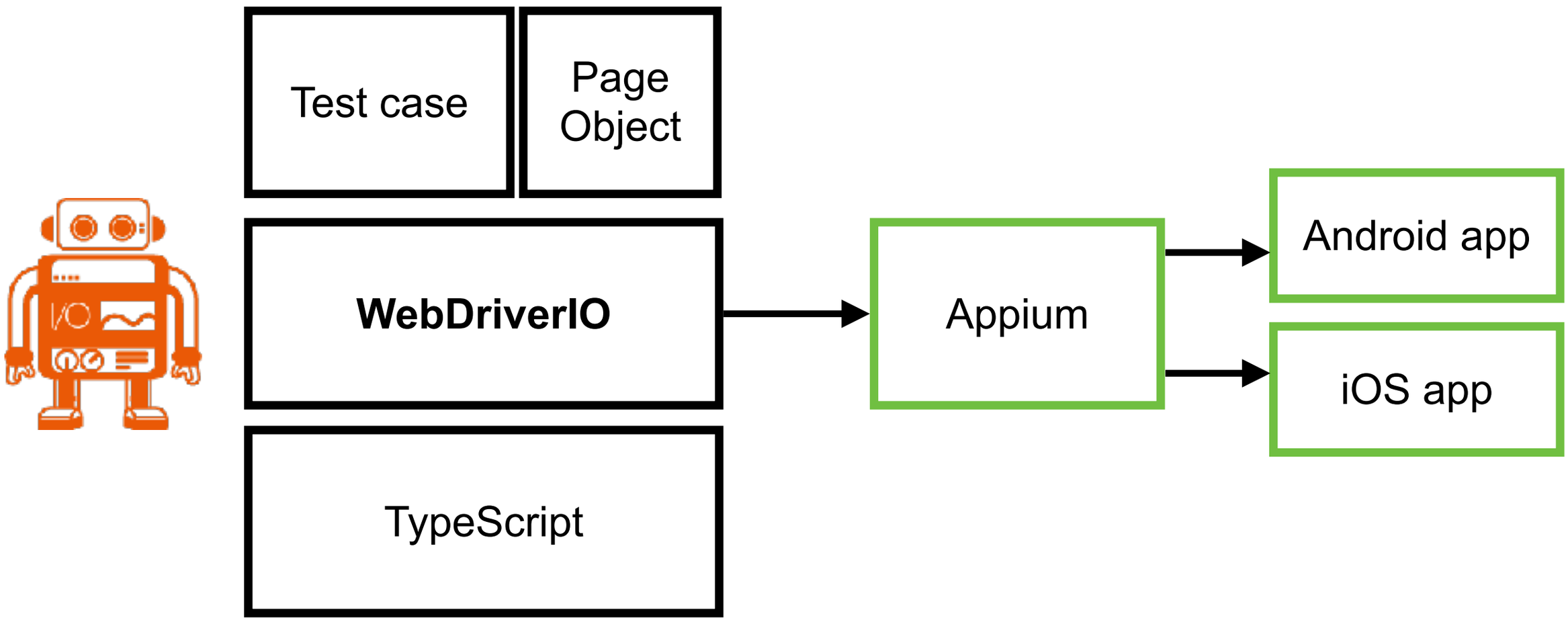
Locator and Value ?

Flutter semantic	iOS	Android
Text	Accessibility-id Text	Resource-id Content-desc
Button	Accessibility-id Text	Resource-id Content-desc
TextField	Accessibility-id Value	Resource-id Value
ListView	Accessibility-id	Resource-id

Write Test Cases

Create Test Project

$npm init wdio@latest .

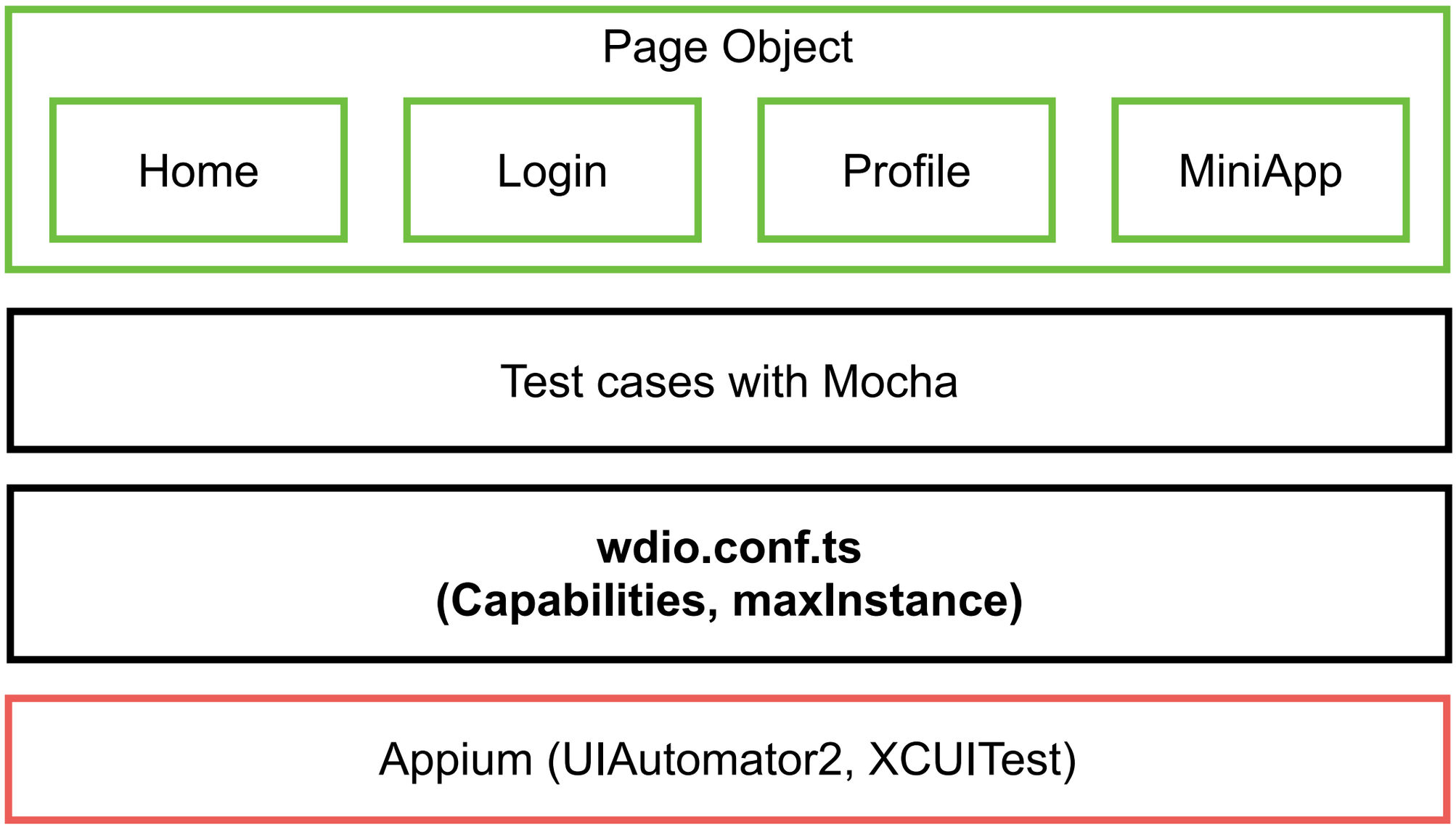


https://webdriver.io/docs/gettingstarted

Structure of project

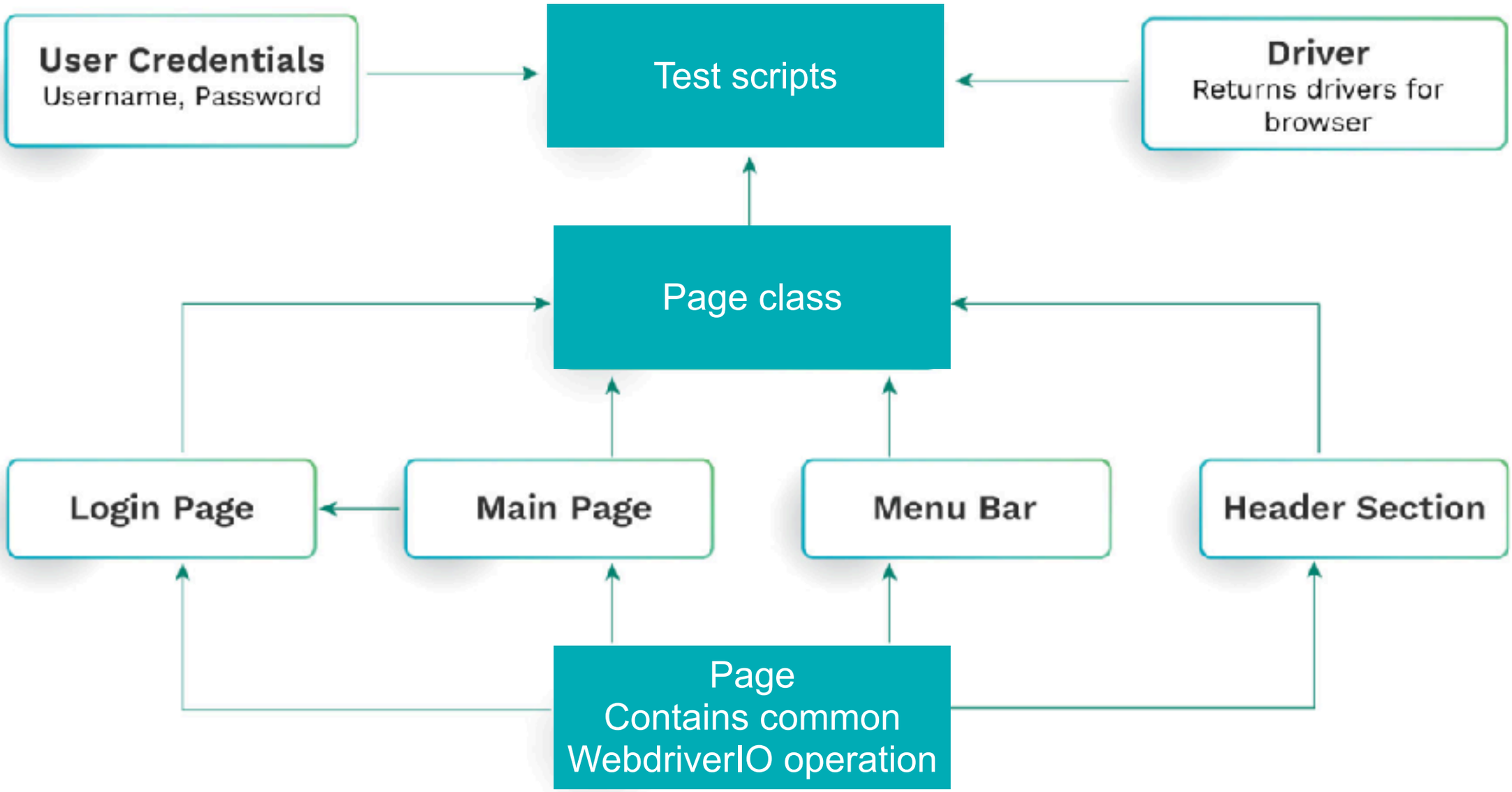
File/Folder name	Description
wdio.conf.ts	ไฟล์ config หลักของ WebDriverIO
test/specs	จัดเก็บ test case ในรูปแบบของ Mocha
test/pageobjects	จัดเก็บ action, assert ต่าง ๆ ของแต่ละ page ตามแนวคิดของ POM ซึ่งถูกเรียกจาก test case
test/utils	จัดเก็บไฟล์ util หรือตัวช่วยของการทดสอบ ในตัวอย่างจะเป็นตัวจัดการ locator ของ ios/android
allure-results	จัดเก็บ test report ในรูปแบบของ Allure

Test Project Structure



https://github.com/up1/workshop-mobile-testing-with-typescript/tree/main/appium_testing

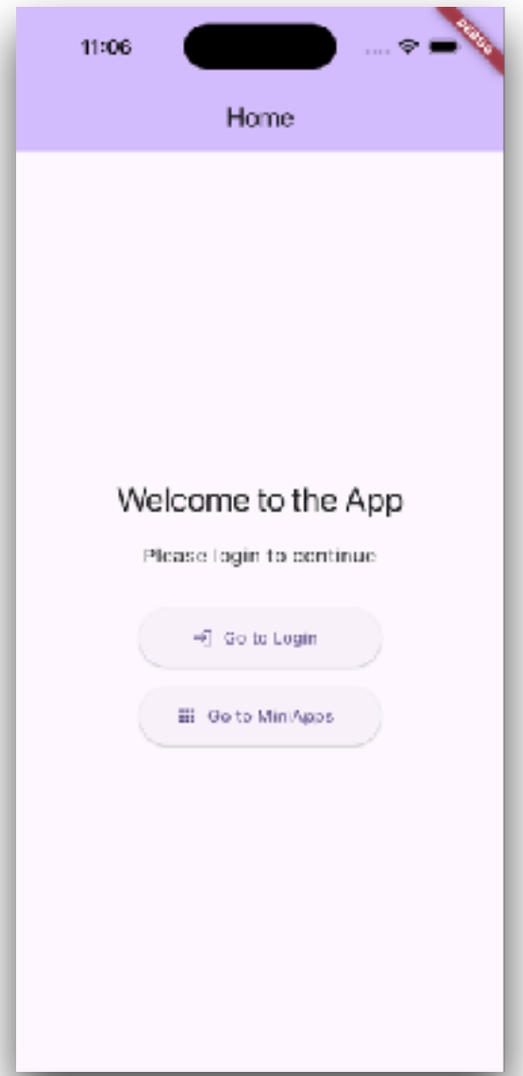
POM (Page Object Model)



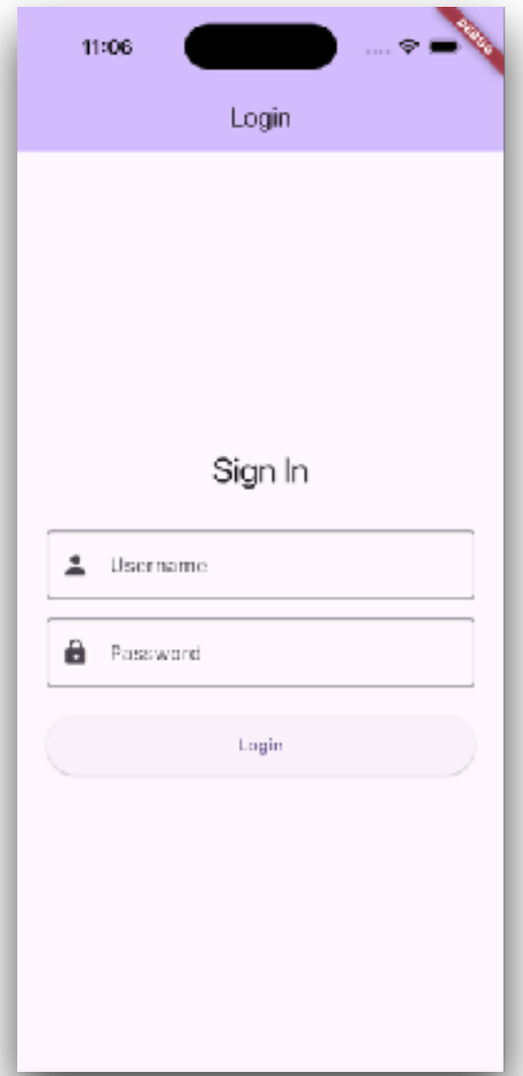
Mobile App for Testing

Flow of Mobile app (1)

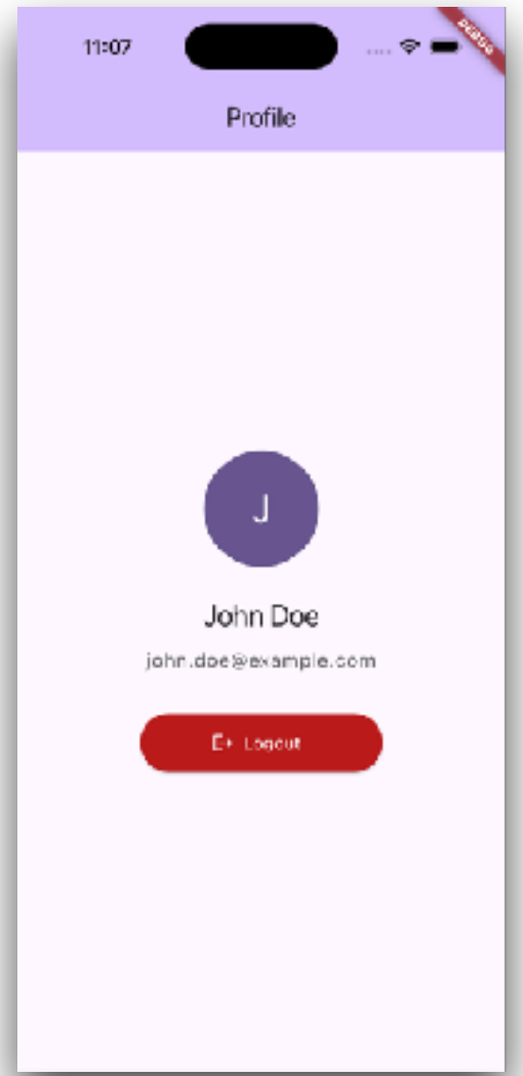
Home



Login

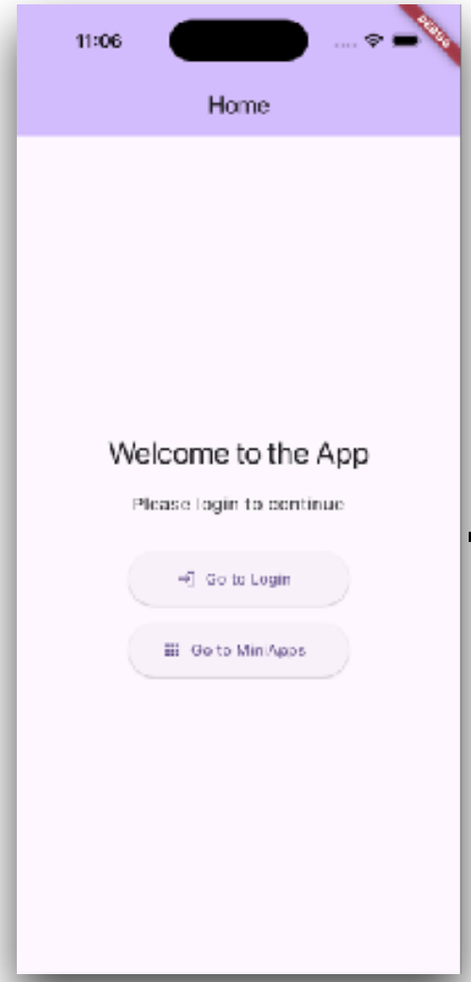


Profile

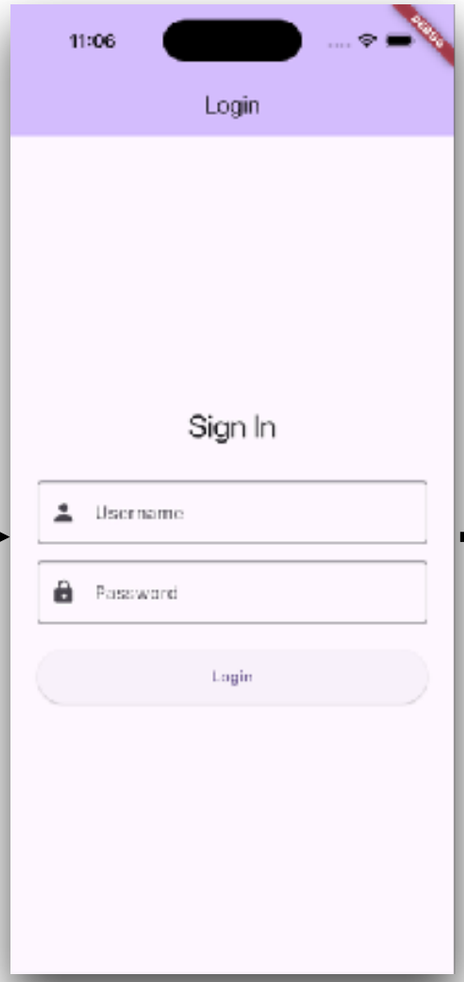


Flow of Mobile app (2)

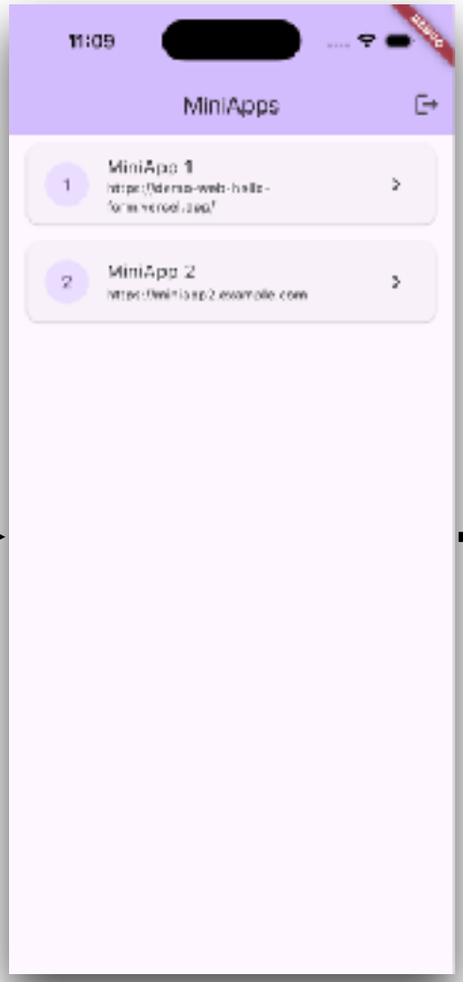
Home



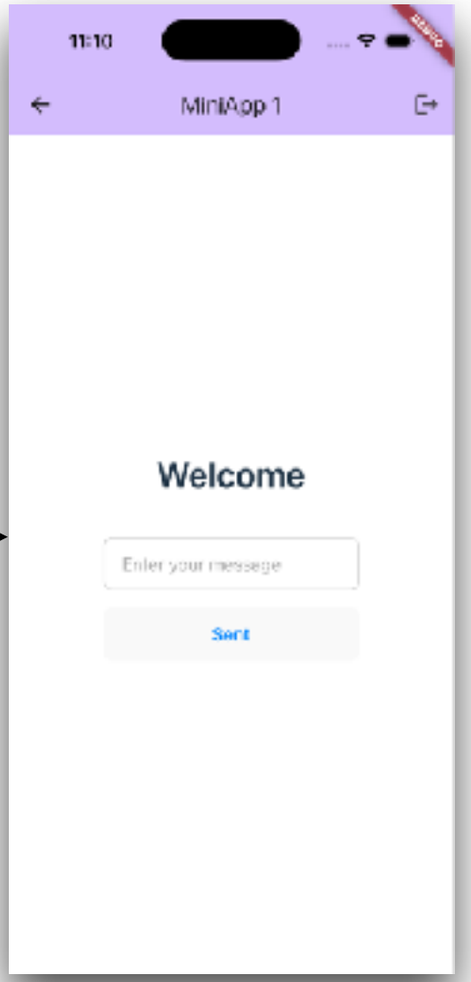
Login



MiniAppList

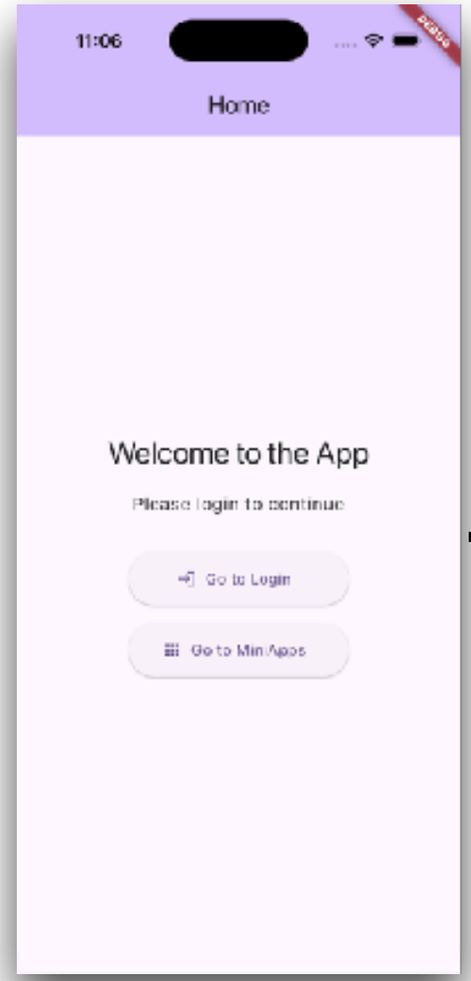


WebView

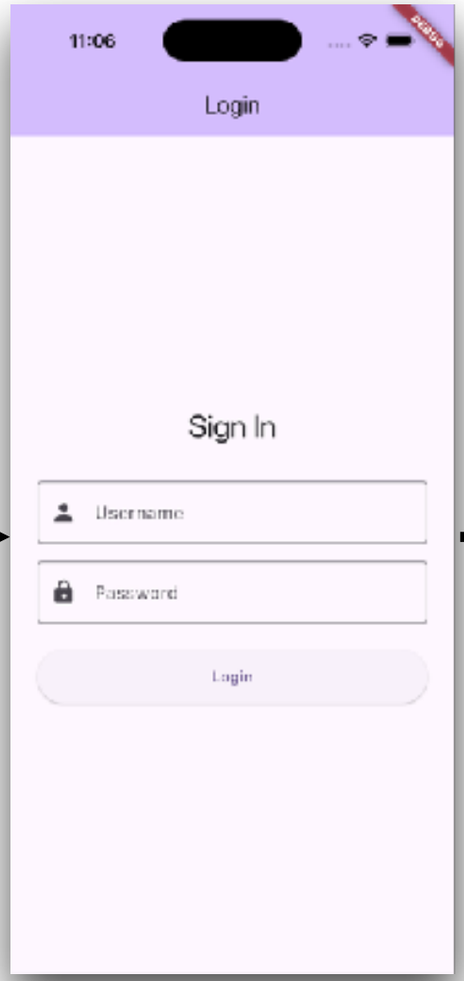


Working with web on mobile

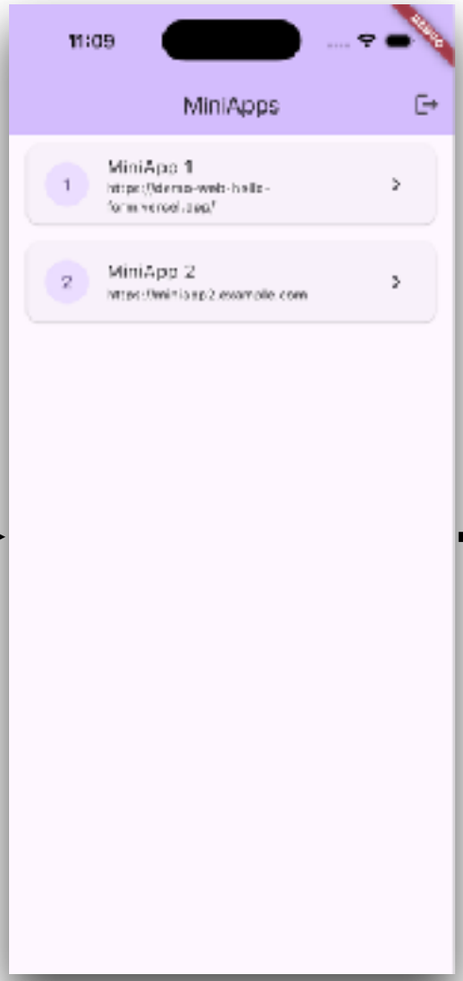
Home



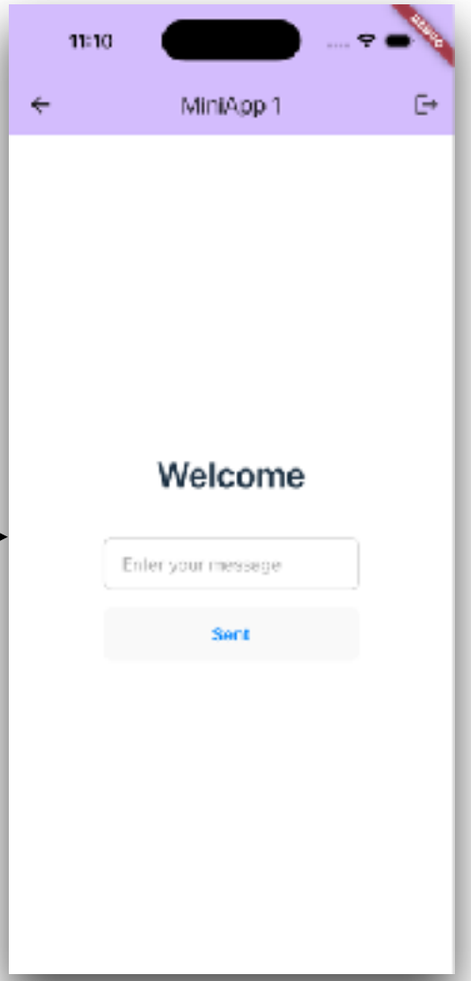
Login



MiniAppList



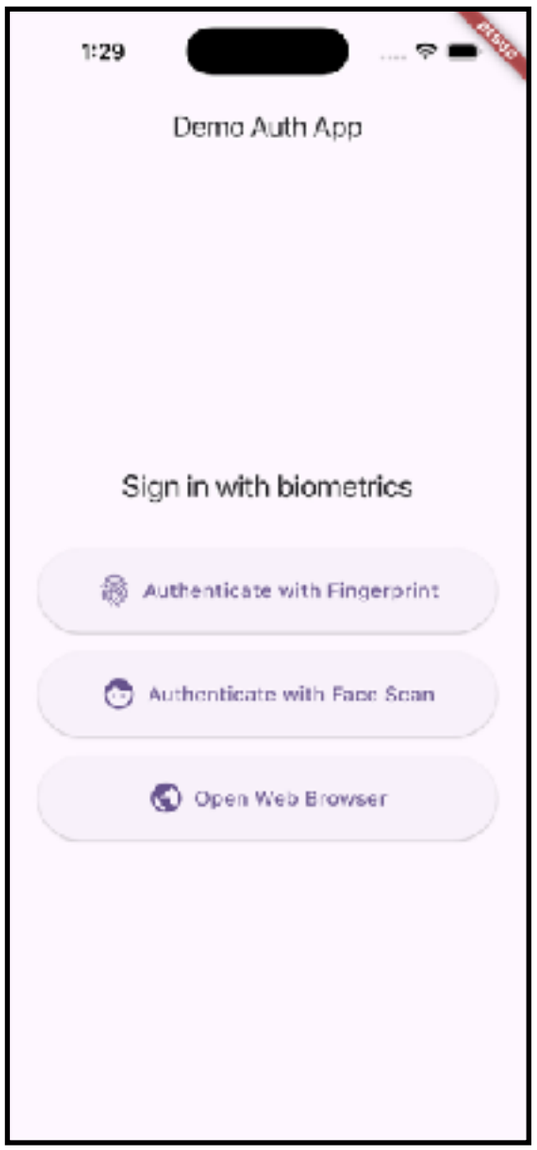
Browser App



Biometric
(Face ID and Fingerprint)

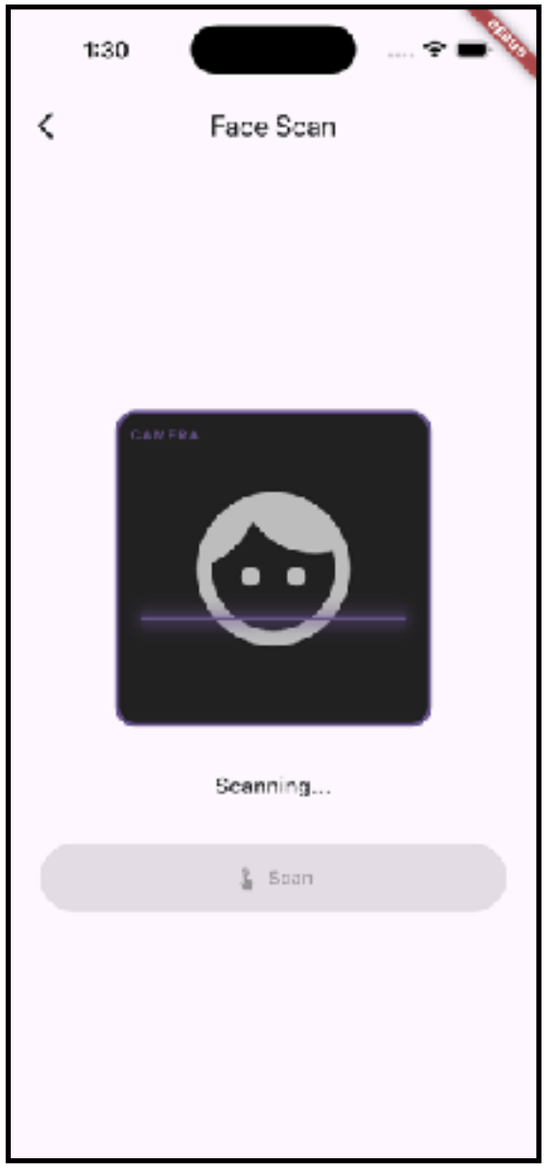
Flow

Home

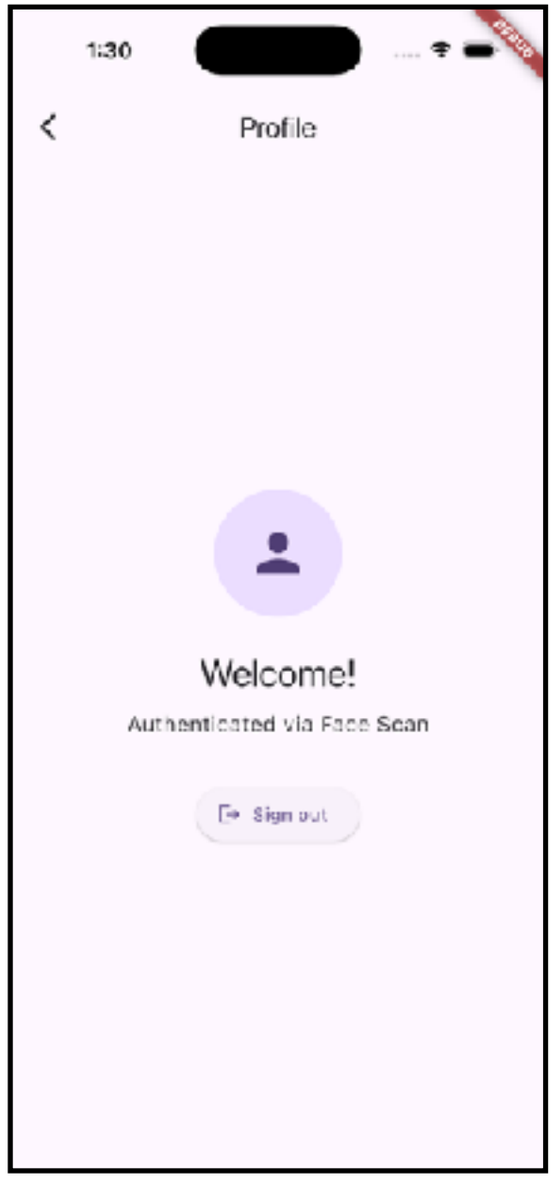


Auth

Auth

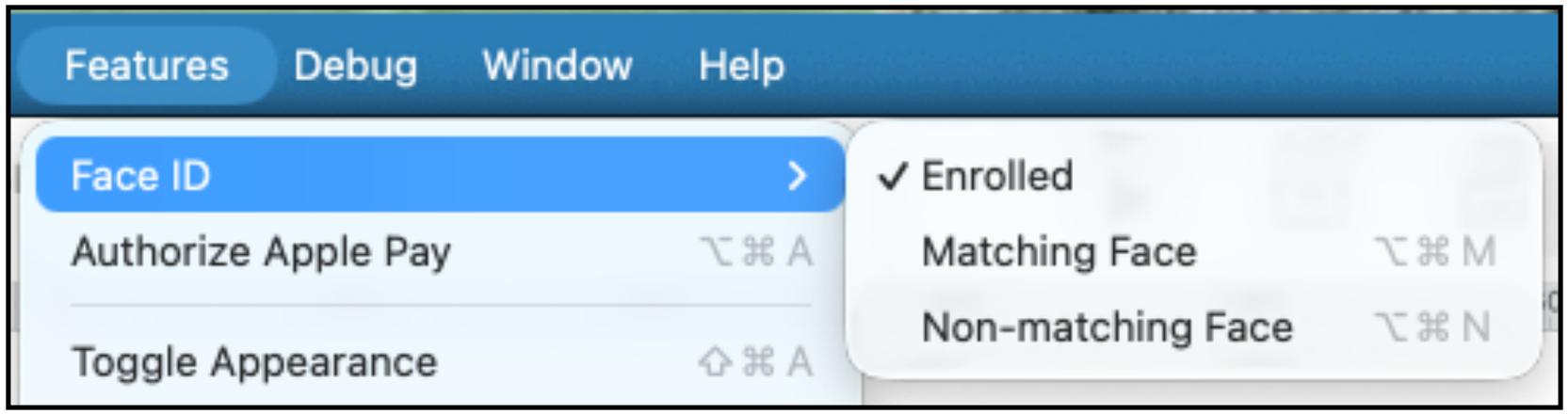


Profile



Biometric Testing (iOS)

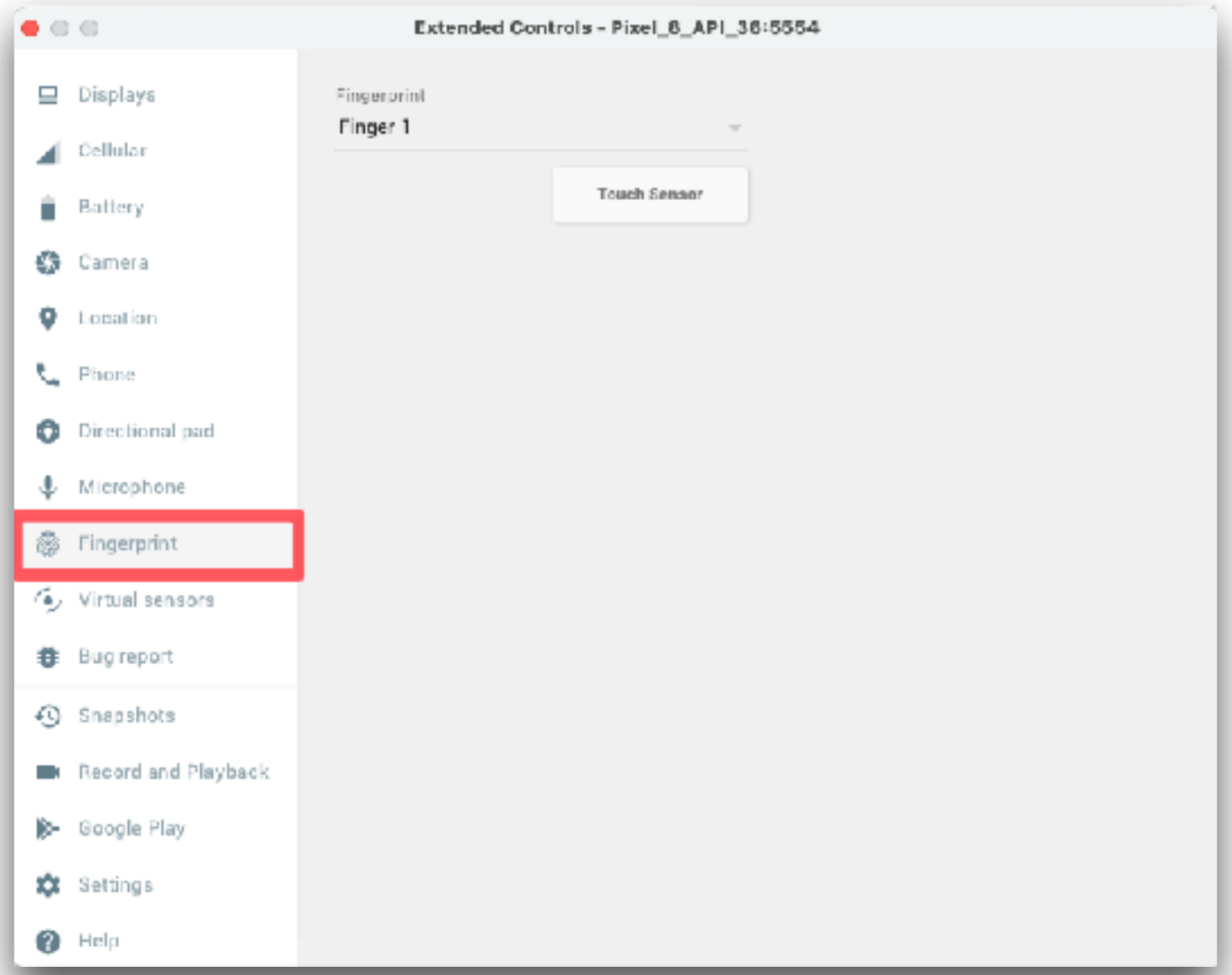
Enrolled Face ID in Simulator



https://webdriver.io/docs/api/mobile/touchId

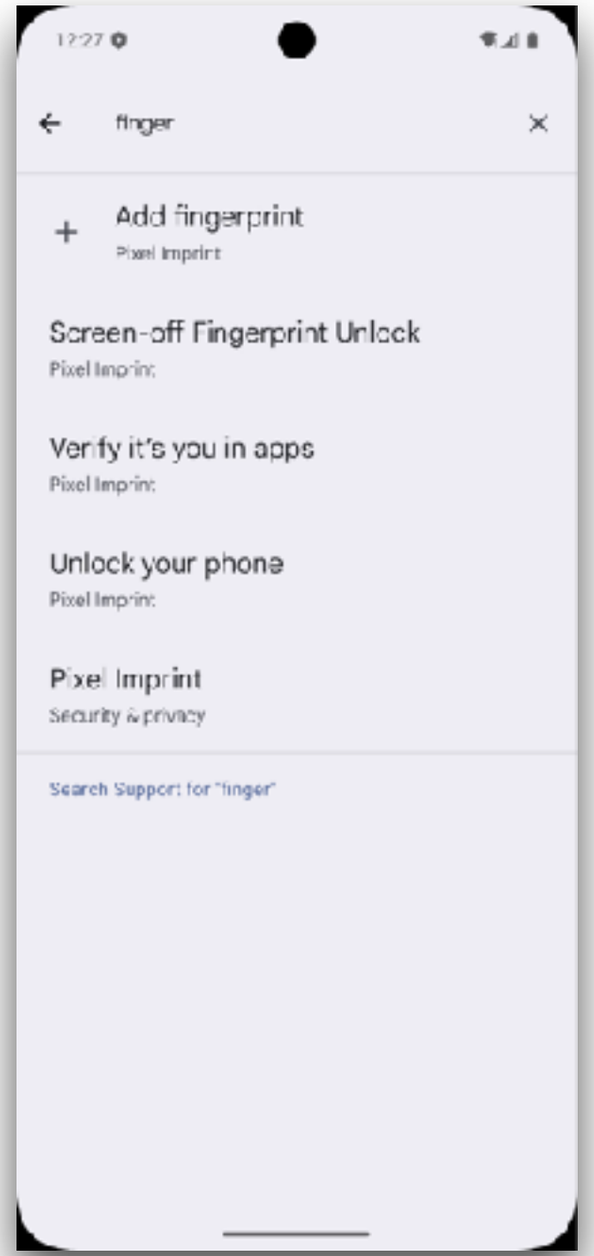
Biometric Testing (Android)

Enable Fingerprint in emulator



Biometric Testing (Android)

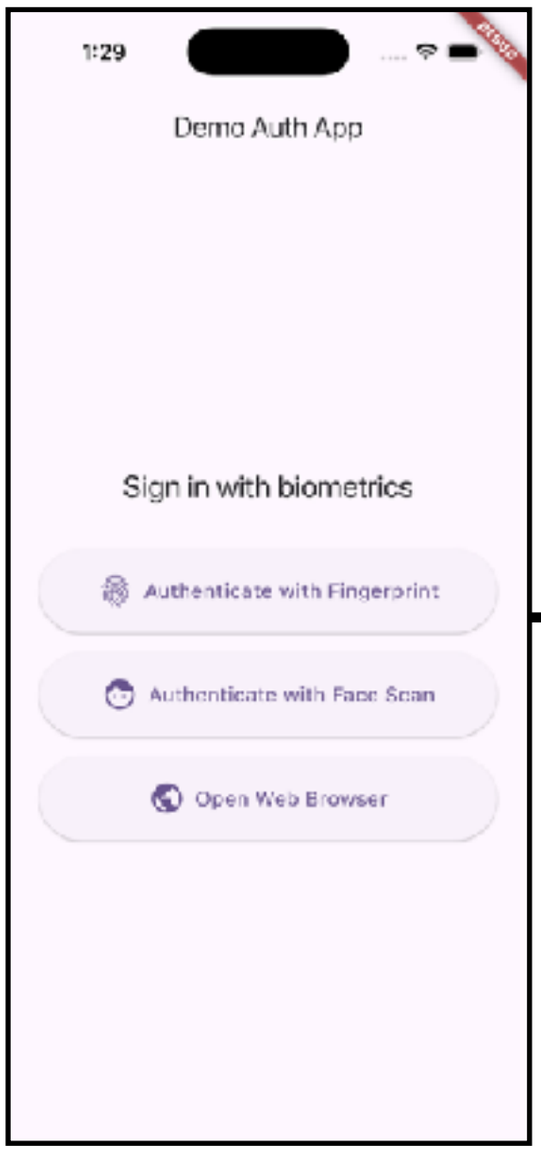
Enable Fingerprint in emulator



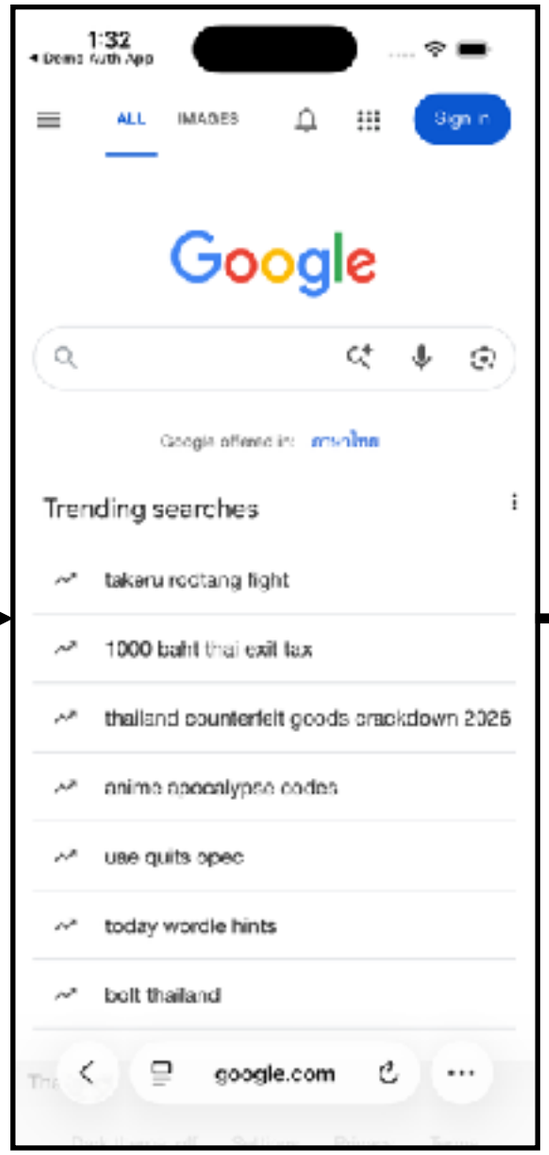
Working with WebView

Flow

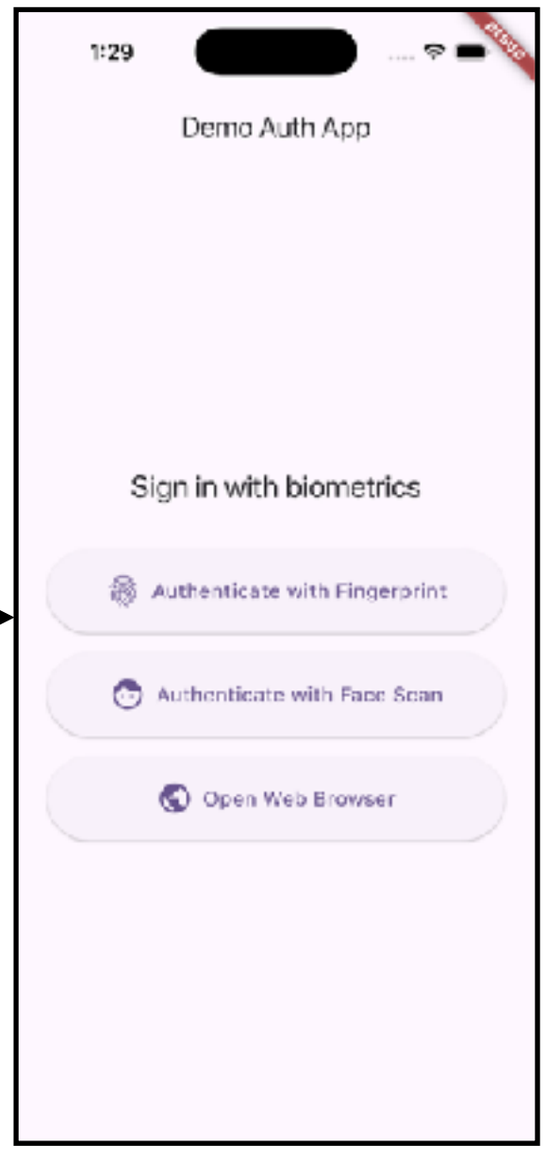
Native



WebView



Native

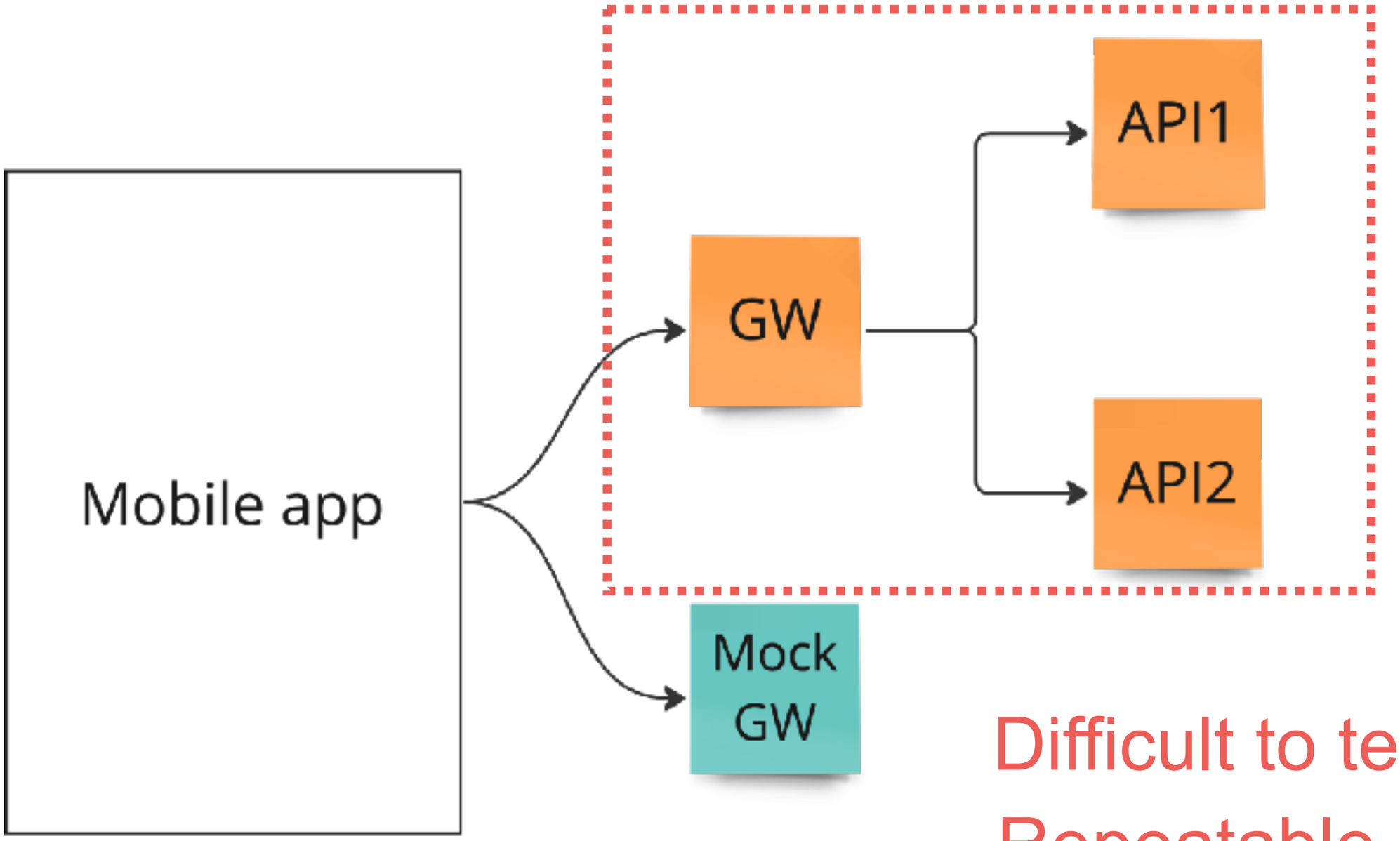


Test strategy ?

Good Tests ?

Fast
Isolate
Repeatable
Self-verify
Timely

Integration Testing



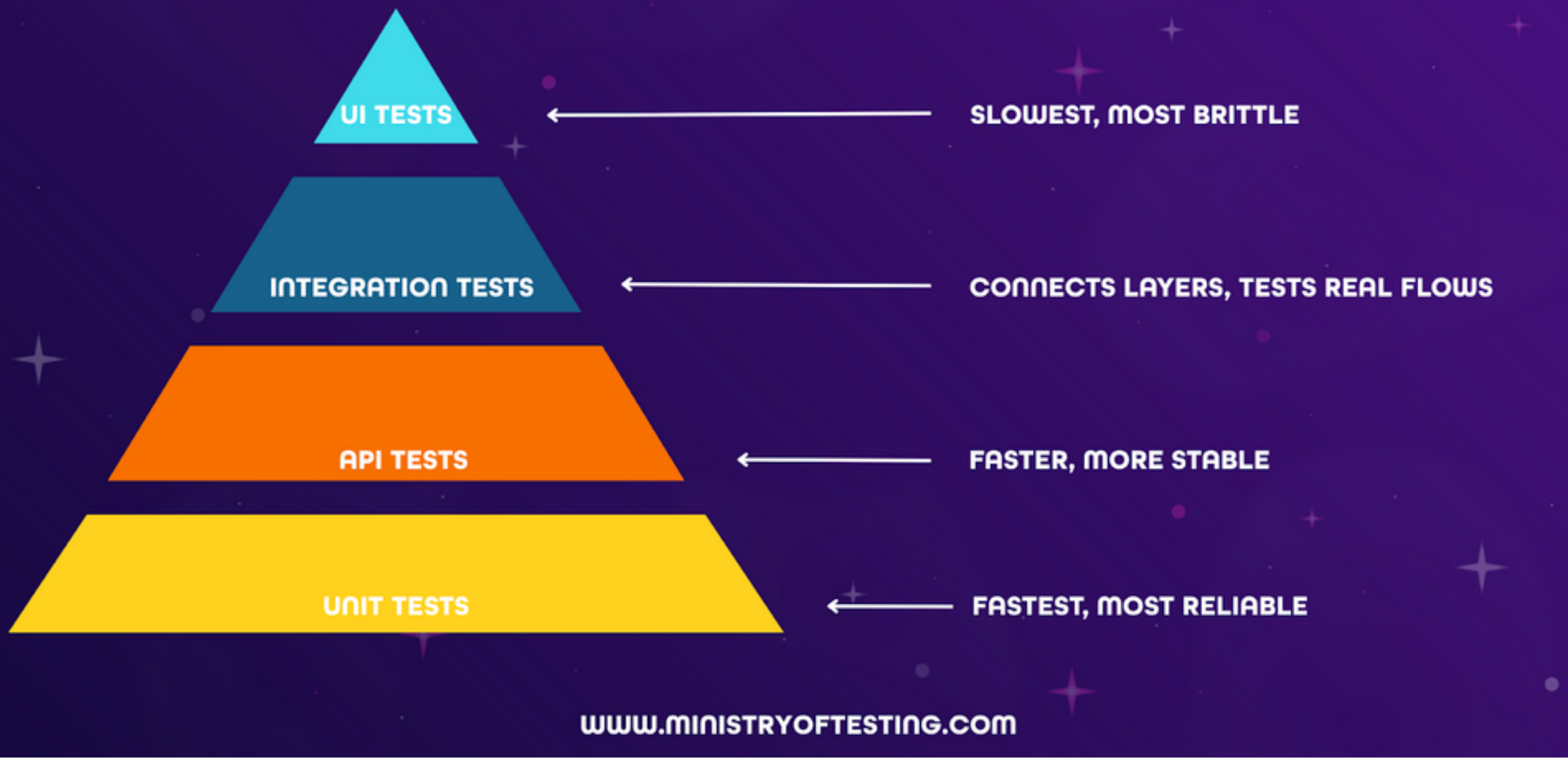
Difficult to test
Repeatable
Reliable

Best Practices for UI testing

Implement Test Automation Pyramid
Prioritize critical user's flows
Use design patterns (Page Object Model)
Avoid static waits (sleep !!)
Integrate into CI/CD pipeline
Ensure test independence

Design for Testability

THE TEST AUTOMATION PYRAMID



WWW.MINISTRYOFTESTING.COM

Common problems in UI testing

Handoff culture where Dev and QA work in Silo

Brittle selectors

Flaky tests

Maintenance overload

Knowledge silo

Real UI not match with original design

Brittle selectors

Root cause

Test rely on unstable attributes like CSS class, XPath
When change UI that effect to test

Solution

Dev and QA collaboration
Use stable element attributes for testing (id, data test id)
Healing test !!

Flaky tests

Root cause

Tests fail intermittently due to timing issues or element not ready for interaction (random fails)

Solution

Wait strategies, don’t use static wait

Use implicit/explicit waits

Manage dependencies and data for testing

Maintenance overload

Root cause

QA spends 40% of time to fixing broken tests

Solution

Shift-left testing and refactor

QA involve early from UI design process

Regularly prune and refactor test case together

Core Practices to Bridge the Gap

Design for testability
Unified testing language for Dev/QA/PO
Common tooling, share result dashboard
Collaboration in planning phase

Clarify acceptance criteria and identify automation bottlenecks before coding start

API Testing with Playwright

https://playwright.dev/docs/api-testing

Q/A